4OR-Q J Oper Res (2013) 11:101–150
DOI 10.1007/s10288-013-0242-3

INVITED SURVEY

Combining metaheuristics with mathematical programming, constraint programming and machine learning

El-Ghazali Talbi

Received: 13 March 2013 / Revised: 17 May 2013 / Published online: 17 July 2013
© Springer-Verlag Berlin Heidelberg 2013

Abstract During the last years, interest on hybrid metaheuristics has risen considerably in the field of optimization and machine learning. The best results found for many optimization problems in science and industry are obtained by hybrid optimization algorithms. Combinations of optimization tools such as metaheuristics, mathematical programming, constraint programming and machine learning, have provided very efficient optimization algorithms. Four different types of combinations are considered in this paper: (i) Combining metaheuristics with complementary metaheuristics. (ii) Combining metaheuristics with exact methods from mathematical programming approaches which are mostly used in the operations research community. (iii) Combining metaheuristics with constraint programming approaches developed in the artificial intelligence community. (iv) Combining metaheuristics with machine learning and data mining techniques.

Keywords Hybrid metaheuristics · Matheuristics · Mathematical programming · Machine learning · Constraint programming · Data mining

Mathematics Subject Classification 90-02

1 Introduction

This paper deals with the development of hybrid metaheuristics to solve difficult and complex optimization problems. A taxonomy of hybrid optimization algorithms is presented in an attempt to provide a common terminology and classification mechanisms. The goal of the general taxonomy given here is to provide a mechanism to allow

E.-G. Talbi (✉)
CNRS, INRIA, University of Lille 1, Lille, France
e-mail: talbi@lifl.fr

comparison of hybrid optimization algorithms in a qualitative way. In addition, it is hoped that the categories and their relationships to each other have been chosen carefully enough to indicate areas in need of future work as well as to help classify future work. The taxonomy proposed here is an extension of the work proposed in Talbi (2002) to deal with combining metaheuristics with other optimization algorithms from mathematical programming (matheuristics), machine learning and constraint programming. The same classification is used for all types of combinations. For each type of hybrid, the main ideas in combining algorithms are detailed. Each class of hybrids is illustrated with some examples. A critical analysis is also carried out.

In fact, the taxonomy could usefully be employed to classify any hybrid optimization algorithm. The basic classification is extended by defining the space of hybrid metaheuristics as a grammar, where each sentence is a method that describes a combination of metaheuristics, mathematical programming, constraint programming and machine learning. This classification can also be applicable to solve multi-objective optimization problems.

The paper is structured as follows. In Sect. 2, our concern is hybrid algorithms combining metaheuristics with complementary metaheuristics. The design and implementation issues of hybrid metaheuristics are detailed. A taxonomy is presented to provide a unifying view of hybrid metaheuristics. A grammar which generalizes the basic hybridization schemes is defined. In Sect. 3, the combination of metaheuristics with mathematical programming approaches is addressed. Section 4 deals with the combination of metaheuristics with constraint programming techniques. Then, in Sect. 5 the combination of metaheuristics with data mining and machine learning algorithms is addressed.

2 Hybrid metaheuristics

Single-solution based algorithms (e.g. local search, simulated annealing) manipulate and transform a single solution during the search while in population-based algorithms (e.g. particle swarm, evolutionary algorithms) a whole population of solutions is evolved. These two families have complementary characteristics: single-solution based metaheuristics are exploitation oriented; they have the power to intensify the search in local regions. Population-based metaheuristics are exploration oriented; they allow a better diversification in the whole search space.

Hybridization of metaheuristics involves a few major issues which may be classified as design and implementation. The former category concerns the hybrid algorithm itself, involving issues such as functionality and architecture. The implementation consideration includes the hardware platform, programming model and environment on which the algorithm is to be run. In this paper, a difference is made between the design issues used to introduce hybridization and implementation issues that depend on the execution model of the algorithms.

2.1 Design issues

The taxonomy will be kept as small as possible by proceeding in a *hierarchical* way as long as possible, and then a *flat classification* is used according to some criteria.

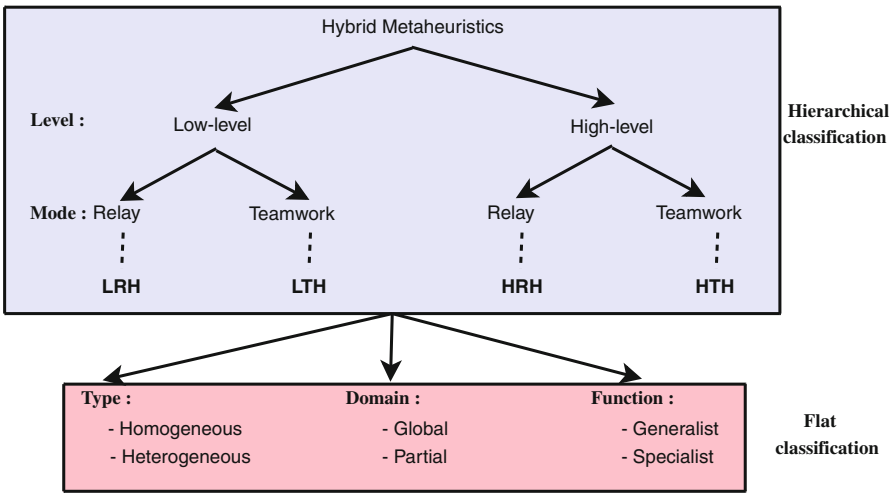


Fig. 1 Classification of hybrid metaheuristics in terms of design issues

2.1.1 Hierarchical classification

The structure of the hierarchical portion of the taxonomy is shown in Fig. 1. A discussion about the hierarchical portion then follows. At the first level, one may distinguish between *low-level* and *high-level* hybridizations. The low-level hybridization addresses the functional composition of a single optimization method. In this hybrid class, a given internal function of a metaheuristic is replaced by another optimization method. In high-level hybrid algorithms, the different optimization methods are self-contained. There is no direct relationship to the internal workings of a metaheuristic.

In relay hybridization, a set of metaheuristics is applied one after another, each using the output of the previous as its input, acting in a pipeline fashion. Teamwork hybridization represents cooperative optimization models, in which many cooperating agents evolve in parallel; each agent carries out a search in a solution space and may cooperate with the other agents.

Four classes are derived from this hierarchical taxonomy:

LRH (Low-level relay hybrid): This class of hybrids represents algorithms in which a given metaheuristic is embedded into a S-metaheuristic (Single solution based metaheuristic) Talbi (2009). Few examples of hybrid metaheuristics belong to this class.

Example 1 (Embedding local search into simulated annealing) The main idea is to incorporate deterministic local search techniques into simulated annealing so that the Markov chain associated to simulated annealing explores only local optima (Martin et al. 1992). The algorithm proceeds as follows: suppose the configuration is currently locally optimal. This is labeled `Start` in Fig. 2. A perturbation or a "kick" is applied to this configuration which significantly changes the current solution `Start`. After the kick, the configuration labeled `Intermediate` in the figure is reached. It is much better to first improve `Intermediate` by a local search and apply the

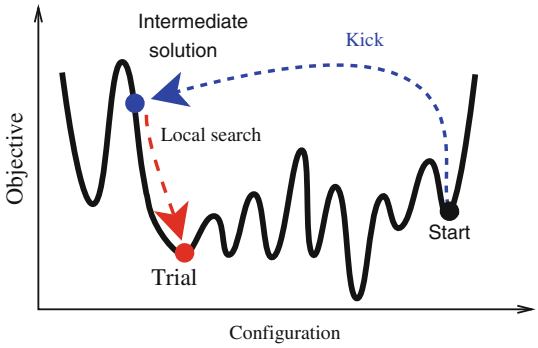


Fig. 2 An example of LRH hybridization embedding local search into simulated annealing. The figure gives a schematic representation of the objective function and the configuration modification procedure used in the LRH hybrid algorithm

accept/reject test of simulated annealing only afterwards. The local search takes us from `Intermediate` to the configuration labeled `Trial`, and then the accept/reject test is applied. If `Trial` is accepted, one has to find an interesting large change to `Start`. If `Trial` is rejected, return to `Start`. Many of the barriers (the "ridges") of the fitness landscape are jumped over in one step by the hybrid metaheuristic.

To implement the above hybridization, the choice for an appropriate "kick" should be adapted to both the optimization problem and the local search method used. For the traveling salesman problem, if the local search algorithm used is the 2-opt local search heuristic, the "kick" move must apply a k-change with $k > 2$ to prevent cycles. The "kick" operator must attain solutions which are always outside the neighborhood associated to the local search algorithm.

LTH (Low-level teamwork hybrid): Two competing goals govern the design of a metaheuristic: exploration and exploitation. Exploration is needed to ensure that every part of the space is searched enough to provide a reliable estimate of the global optimum. Exploitation is important since the refinement of the current solution will often produce a better solution. P-metaheuristics (Population based metaheuristics) Talbi (2009) (e.g. evolutionary algorithms, scatter search, particle swarm, ant colonies) are powerful in the exploration of the search space, and weak in the exploitation of the solutions found.

Therefore, most efficient P-metaheuristics have been coupled with S-metaheuristics such as local search, simulated annealing and tabu search, which are powerful optimization methods in terms of exploitation. The two classes of algorithms have complementary strengths and weaknesses. The S-metaheuristics will try to optimize locally, while the P-metaheuristics will try to optimize globally. In LTH hybrid, a metaheuristic is embedded into a P-metaheuristic[1] (Fig. 3). This class of hybrid algorithms is very popular and has been applied successfully to many optimization problems. Most of the state-of-the art of P-metaheuristics integrate S-metaheuristics.

Example 2 (Embedding S-metaheuristics into evolutionary algorithms) When an evolutionary algorithm is used, its standard operators may be augmented with the ability to perform local search. Instead of using a blind operator acting regardless of the fitness

[1] This class of hybrid metaheuristics includes *memetic algorithms*.

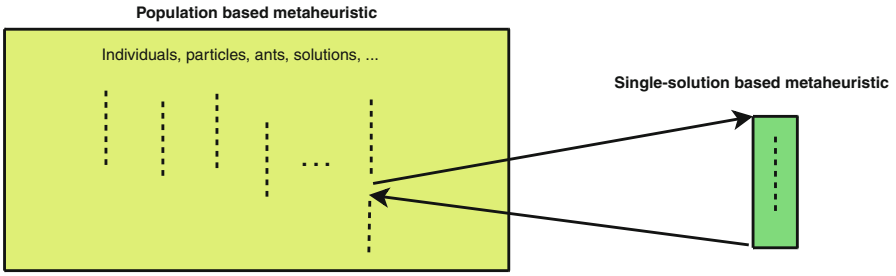


Fig. 3 Low-level teamwork hybrid (LTH). S-metaheuristics are embedded into P-metaheuristics

of the original individual and the operated one, an operator which is a heuristic that considers an individual as the origin of its search applies itself, and finally replaces the original individual by the enhanced one (see Fig. 4). The use of local search with evolutionary algorithms is also inspired by biological models of learning and evolution. Evolutionary algorithms (EAs) take many cues from mechanisms observed in natural evolution. Similarly, models of learning are often equated with techniques for local optimization (Rumelhart et al. 1986). Research on the interaction between evolution and learning had naturally led computer scientists to consider interactions between evolutionary algorithms and local optimization (Belew et al. 1991).

The genetic operators replaced or extended are generally mutation[2] and crossover.

- **Mutation:** The local search algorithm may be a simple local search (Suh and Van Gucht 1987; Ulder et al. 1990; Jog et al. 1989), tabu search (Fleurent and Ferland 1994; Kim et al. 1995; Thiel and Voss 1994), simulated annealing algorithm (Chen and Flann 1994; Wang and Kao 1995) or any S-metaheuristic (e.g. threshold accepting, guided local search). This kind of operators is qualified *lamarckian*.[3] In the lamarckian model, an individual is replaced by the local optimum found, contrary to the *baldwin* model where the local optimum is just used to evaluate the individual. In several occasions, LTH has provided better results than other methods on difficult problems. For instance, good results have been obtained on the graph coloring problem combining a genetic algorithm with tabu search (Fleurent and Ferland 1996). A local search algorithm which uses problem-specific knowledge may be incorporated into the genetic operators (Chu 1997). Questions concerning the best use of local search with a genetic algorithm have been addressed in Hart (1994).
- **Crossover:** Classical crossover operators do not use any heuristic information about a specific application domain. They are blind operators. One can introduce heuristic crossover in order to account for problem-specific information (Grefenstette 1987). For instance, greedy heuristics for the crossover operator have shown to improve EAs results when applied to job-shop scheduling, set covering, and traveling salesman problems (Davis 1985).

[2] Also known as evolutionary local search algorithms.

[3] The name is an allusion to Jean Batiste de Lamarck's contention that phenotype characteristics acquired during lifetime can become heritable traits.

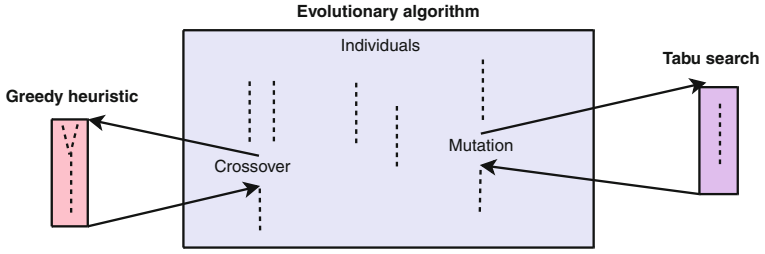


Fig. 4 Illustration of a LTH hybrid. For instance, a tabu search is used as a mutation operator and a greedy heuristic as a crossover operator into an evolutionary algorithm

This hybrid model can be used to improve any P-metaheuristic: ant colonies (Taillard and Gambardella 1997; Stutzle and Hoos 1997), genetic programming (O'Reilly and Oppacher 1995), particle swarm optimization, and so forth. The S-metaheuristic has been introduced to intensify the search. Let us notice that the scatter search metaheuristic already includes an improvement procedure which is based on S-metaheuristics (Cung et al. 1997).

The main problem encountered in this class of hybrids is *premature convergence*. Indeed, if the hybridization is applied at each iteration, very competitive solutions will be generated at the beginning of the search which will cause an eventual premature convergence. Conditional hybridization is carried out to prevent this phenomenon by applying the combination:

- **Static manner:** for instance the combination is performed at a given frequency. The hybridization is applied once for a given number of iterations.
- **Adaptive manner:** when a given event occurs during the search the hybridization is performed. For instance, if there is no improvement of the search for a given number of iterations.

HRH (High-level relay hybrid): In HRH hybrids, self-contained metaheuristics are executed in a sequence. For example, the initial solution of a given S-metaheuristic may also be generated by another optimization algorithm. Indeed, the initial solution in S-metaheuristics has a great impact on their performances. A well known combination scheme is to generate the initial solution by greedy heuristics which are in general of less computing complexity than iterative heuristics (Fig. 5).

This scheme may be also applied to P-metaheuristics, but a randomized greedy heuristic may also be applied to generate a diverse population (Fig. 5). Greedy heuristics are in general deterministic algorithms and then they always generate the same solution. On the other hand, the diversity of the initial population has a great impact on the performance of P-metaheuristics. This hybrid scheme is carried out explicitly in the scatter search metaheuristic.

Combining in the HRH scheme P-metaheuristics with S-metaheuristic is also largely applied. It is well known that P-metaheuristics are not well suited for fine-tuning structures which are very close to optimal solutions. Indeed, the strength of P-metaheuristics is in quickly locating the high performance regions of vast and complex search spaces. Once those regions are located, it may be

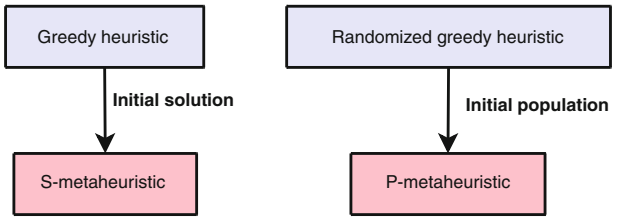


Fig. 5 High-level relay hybridization. (*Left*) Generation of the initial solution of a S-metaheuristic by a greedy algorithm. (*Right*) Generation of the initial population of a P-metaheuristic by a randomized greedy heuristic

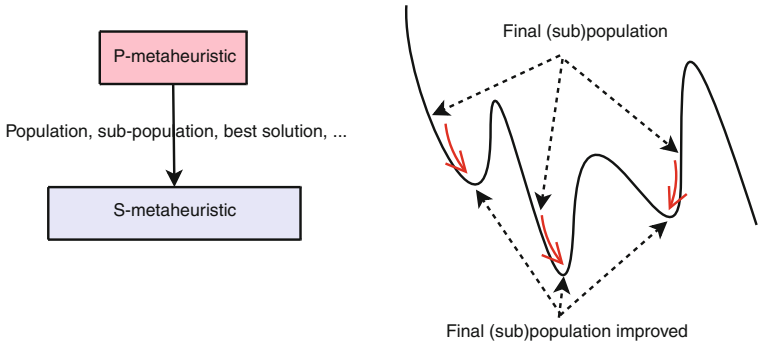


Fig. 6 High-level relay hybridization. There may be more than two algorithms to be pipelined

useful to apply S-metaheuristics to the high performance structures evolved by the P-metaheuristic.

A fundamental practical remark is that after a certain amount of time, the population is quite uniform and the fitness of the population is no longer decreasing. The odds to produce fitter individuals are very low. That is, the process has fallen into a *basin of attraction* from which it has a low probability to escape.

In order to find the optimal points in basins of attraction as efficiently as possible, previously found basins of attractions can be exploited. It is experimentally clear that this exploitation may be more efficiently performed by another algorithm than by a P-metaheuristic. Hence, it is much more efficient to use a S-metaheuristic such as a hill-climbing or tabu search (see Fig. 6). The HRH hybridization may use a greedy heuristic to generate a good initial population for the P-metaheuristic (see Fig. 6). At the end of a simulated annealing search, it makes sense to apply local search on the best found solution to ensure that it is a local optimum.

In this hybrid scheme, the S-metaheuristics may be applied to:

- **The whole population:** this will lead to the best final solutions but with a more important computational cost of the search.
- **A sub-population:** the selection of the subpopulation may be based on the diversity of the solutions. This is a good compromise between the complexity of the search and the quality of the final results.

– **The best solution of the population:** the S-metaheuristic is applied once on the best solution of the obtained population. This procedure will reduce the search time but does not ensure to find the best solution.

A *path relinking* strategy may be applied to a population or a set of elite solutions found by a metaheuristic (Aiex et al. 2003). Path relinking may be seen as an intensification task over a given population of solutions.

Example 3 (HRH hybrid evolutionary algorithms) many papers in the literature have used the idea of HRH hybridization for EAs. In Mahfoud and Goldberg (1995), Talbi et al. (1994), the considered hybrid scheme introduces respectively simulated annealing and tabu search to improve the population obtained by a genetic algorithm (GA). In Nissen (1994), the author introduces hill-climbing to improve the results obtained by an evolution strategy (ES). In Lin et al. (1991), the proposed algorithm starts from simulated annealing and uses GAs to enrich the obtained solutions. Experiments performed on the graph partitioning problem using the tabu search algorithm exploiting the result found by a GA give better results than a search performed either by the GA, or the tabu search alone (Talbi et al. 1994).

HTH (High-level teamwork hybrid): The HTH[4] scheme involves several self-contained algorithms performing a search in parallel, and cooperating to find an optimum. Intuitively, HTH will ultimately perform at least as well as any of the cooperating algorithms.

Example 4 (Island model for genetic algorithms) The first HTH hybrid model has been proposed for genetic algorithms (GAs). This is the well known island model.[5] The population in this model is partitioned into small subpopulations by geographic isolation. A GA evolves each subpopulation, and individuals can migrate between subpopulations (Fig. 7). This model is controlled by several parameters: the topology that defines the connections between subpopulations, the migration rate that controls the number of migrant individuals, the replacement strategy used, and a migration interval that affects how often migration occurs. In some island models, the individuals really migrate and therefore leaves empty space in the original population. In general, the migrated individuals remain in the original population (i.e. pollination model Sprave 1999).

Let us present some pioneering island models for GAs. Tanese proposed a GA based HTH scheme that used a 4-D hypercube topology to communicate individuals from one subpopulation to another (Tanese 1987). Migration is performed at uniform periods of time between neighbor subpopulations along one dimension of the hypercube. The migrants are chosen probabilistically from the best individuals of the subpopulation and they replace the worst individuals in the receiving subpopulation.

Cohoon et al. (1987) proposed a HTH based on the theory of "punctuated equilibria". A linear placement problem was used as a benchmark and experimented using a

[4] HTH hybrids is referred as *multiple interacting walks* Verhoeven and Aarts (1995) *multi-agent algorithms* (Boese 1996), and *cooperative search algorithms* (Clearwater et al. 1991, 1992; Hogg and Williams 1993; Huberman 1990; Toulouse et al. 1996).

[5] Also known as migration model, diffusion model, and coarse grain model.

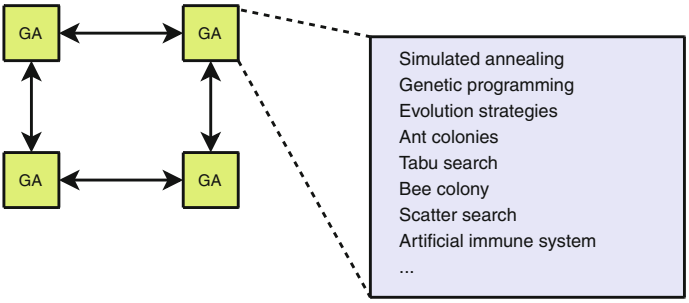


Fig. 7 The island model of genetic algorithms as an example of High-level teamwork hybrid (HTH). The same model has been used with different topologies for simulated annealing, genetic programming, evolution strategy, ant colony, tabu search, bee colony, artificial immune system, etc

mesh topology. They found that the algorithm with migration outperforms the algorithm without migration and the standard GA. This work was later extended using a VLSI design problem (graph partitioning) on a 4-D hypercube topology (Cohoon et al. 1990, 1991).

Belding in Belding (1995) attempted to extend the Tanese's work using the Royal Road continuous functions. Migrant individuals are sent to a randomly selected subpopulation, rather than using a hypercube topology. The global optimum was found more often when migration (i.e. cooperating GAs) occurred than in completely isolated cases (i.e. non-cooperating GAs).

Afterwards, the HTH hybrid model has been generalized to other P-metaheuristics and S-metaheuristics. Indeed, the HTH hybrid model has also been applied to simulated annealing (Falco et al. 1997), genetic programming (Koza and Andre 1995), evolution strategies (Voigt et al. 1990), ant colonies (Mariano et al. 1998), scatter search (Cung et al. 1997), tabu search (Falco et al. 1994), and so on.

2.1.2 Flat classification

Homogeneous/heterogeneous: In homogeneous hybrids, all the combined algorithms use the same metaheuristic. Hybrid algorithms such as the island model for GAs belong to this class of hybrids. The homogeneous metaheuristics may differ in the initialization of their (Fig. 8):

- **Parameters:** In general, different parameters are used for the algorithms. For instance, in the HTH hybrid scheme which is based on tabu search, the algorithms may be initialized with different tabu list sizes (Voss 1993); different crossover and mutation probabilities may be used in evolutionary algorithms, etc.
- **Search components:** Given a metaheuristic, one can use different strategies for any search component of the metaheuristic, such as the representation of solutions, objective function approximations (Eby et al. 1998; Sefraoui and Periaux 2000), initial solutions, search operators (neighborhood, mutation, crossover, ...), termination criteria, etc.

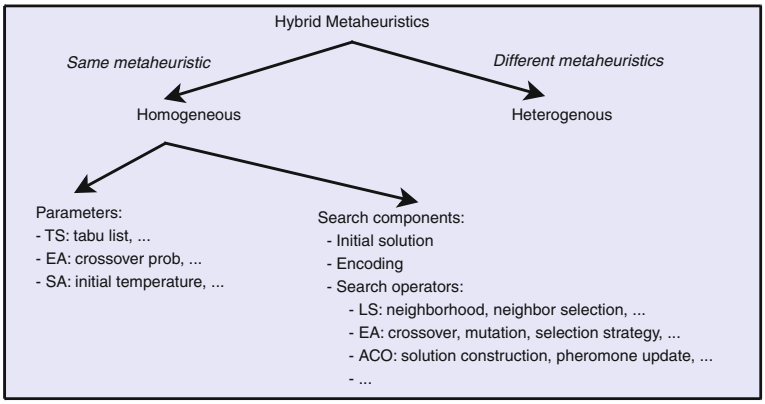


Fig. 8 Homogeneous versus heterogeneous hybrid metaheuristics. Some illustrative examples of parameters and search components are illustrated

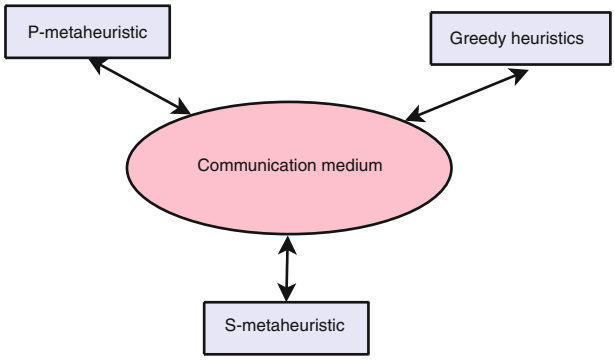


Fig. 9 High-level teamwork hybridization HTH (heterogeneous, global, general). Several search algorithms cooperate, co-adapt, and co-evolve a solution

Using different parameters or search components into a given metaheuristic will increase the robustness of the hybrid algorithm.

Example 5 (Heterogeneous hybrids) In heterogeneous algorithms, different metaheuristics are used (Fig. 9). A heterogeneous HTH algorithm based on genetic algorithms and tabu search has been proposed in Crainic et al. (1997) to solve a network design problem. The population of the GA is asynchronously updated by multiple tabu search algorithms. The best solutions found by tabu search algorithms build an elite population for the GA.

The GRASP method (Greedy Randomized Adaptive Search Procedure) may be seen as an iterated heterogeneous HRH hybrid, in which local search is repeated from a number of initial solutions generated by a randomized greedy heuristic (Feo et al. 1991, 1994). The method is called adaptive because the greedy heuristic takes into account the decisions of the previous iterations (Feo and Resende 1995).

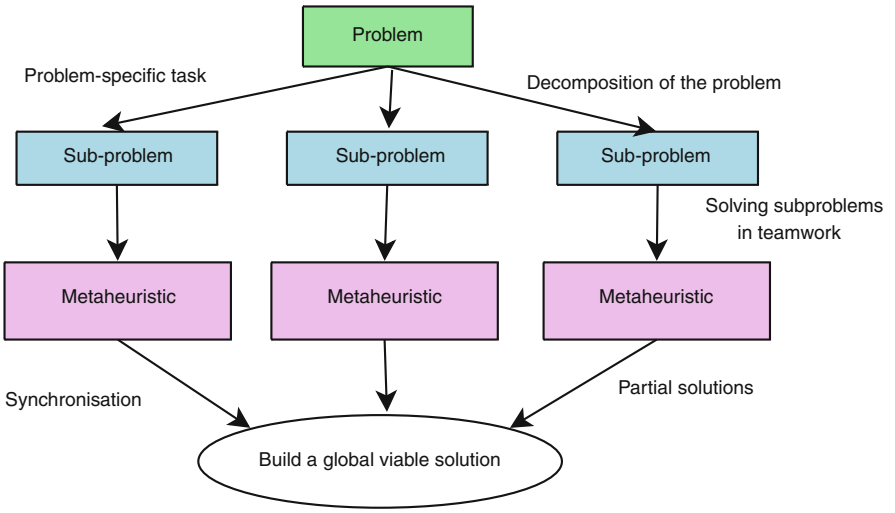


Fig. 10 Partial hybrid schemes. Several search algorithms cooperate in solving sub-problems. A synchronization is performed to build a global solution from the partial solutions found

Global/partial: From another point of view, one can also distinguish two kinds of cooperation: global and partial. In global hybrids, all the algorithms explore the same whole search space. The goal is here to explore the space more thoroughly. All the above mentioned hybrids are *global* hybrids, in the sense that all the algorithms solve the whole optimization problem. A global HTH algorithm based on tabu search has been proposed in Crainic et al. (1995), where each tabu search task performs a given number of iterations, then broadcasts the best solution. The best of all solutions becomes the initial solution for the next phase.

In partial hybrids, the problem to be solved is decomposed a priori into sub-problems, each one having its own search space (Fig. 10). Then, each algorithm is dedicated to the search in one of these sub-spaces. Generally speaking, the sub-problems are all linked with each others, thus involving constraints between optima found by each algorithm. Hence, the algorithms communicate in order to respect these constraints and build a global viable solution to the problem.

Example 6 (Partial hybrids) This approach has been applied to many specific metaheuristics such as simulated annealing and tabu search algorithms (Taillard 1993). It is also a part of more general search framework such as POPMUSIC (Partial OPtimization Metaheuristic Under Special Intensification Conditions (Taillard and Voss 2002) and VNDS (Variable Neighborhood Decomposition Search) (Hansen et al. 2001).

Asynchronous teams (A-Teams) represent a general model for a HTH heterogeneous partial hybrid strategy (Talukdar et al. 1998). They manipulate a set of solutions, which may be global or partial solutions. A set of agents cooperate via a blackboard system, a shared memory structure. An agent may be any search algorithm or operator, which consists in picking a (partial) solution from the blackboard, transforming it and sending back the result.

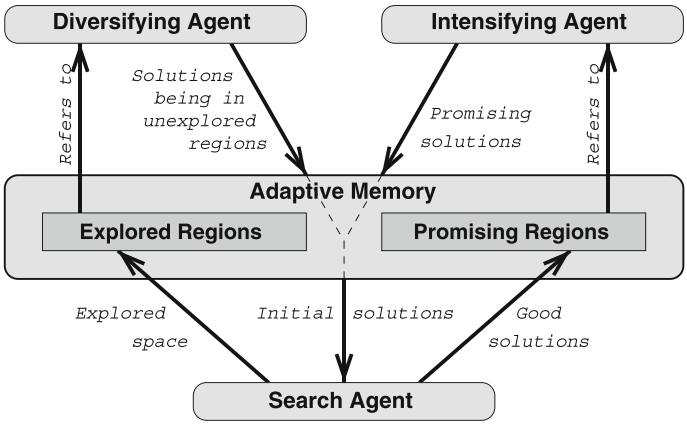


Fig. 11 The COSEARCH HTH specialist hybrid model for metaheuristics. Several search algorithms solve different problems

An example of application of partial homogeneous HTH has been done for the job-shop scheduling problem (Husbands et al. 1990). The search algorithm is a GA. Each GA evolves individuals of a species which represent the process plan for one job. Hence, there are as many cooperating GAs as there are jobs. The communication medium collects the best individuals from each GA, and evaluates the resulting schedule as a whole, rewarding the best process plans.

Decomposition techniques based on partitioning time have been used to solve many problems such as the production lot-sizing (partitioning of time) (Federgruen and Tzur 1999). Decomposition techniques based on partitioning a geographical region have been largely applied to optimization problems associated with Euclidean distances such as the TSP (Karp 1977), the VRP, and the P-median problem (Taillard 2003).

Example 7 (Partitioning a continuous objective function) A function f is separable if:

$$(argmin_{x_1} f(x_1, \ldots), \ldots, argmin_{x_n} f(\ldots, x_n)) = argmin(f(x_1, x_2, \ldots, x_n))$$

It follows that the function f can be optimized in a parallel way using n independent algorithms. Each algorithm will solve a 1-D optimization problem.

Generalist/specialist: All the above mentioned hybrids are *general hybrids*, in the sense that all the algorithms solve the same target optimization problem. *Specialized hybrids* combine algorithms which solve different problems. The COSEARCH generic model belongs to this class of hybrids (Fig. 11). COSEARCH manages the cooperation of a search agent (a local search), a diversifying agent and an intensifying agent. The three agents exchange information via a passive coordinator called the adaptive memory.[6] A main key point of the COSEARCH approach is the design of this memory

[6] The concept of adaptive memory has been proposed in the domain of combinatorial optimization (Taillard et al. 2001). It is similar to the concept of blackboard in the field of Artificial Intelligence (Engelmore 1988).

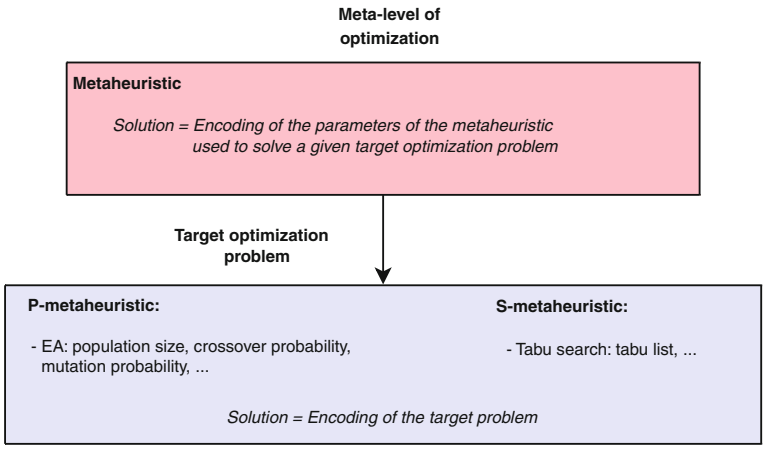


Fig. 12 Meta-level of optimization in metaheuristics. Metaheuristics are used to optimize the parameters of another metaheuristic

which focuses on high quality regions of the search and avoids attractive but deceptive areas. The adaptive memory contains a history of the search. It stores information about the already visited areas of the search space and about the intrinsic nature of the good solutions already found. When diversifying, the local search agent receives starting solutions in unexplored regions. When intensifying, the search agent receives an initial solution in a promising region. The diversifying agent refers to the adaptive memory (information about the explored areas) to yield a solution from an unexplored region. The intensifying agent refers to the adaptive memory (information about promising regions) to produce a promising starting solution.

Example 8 (COSEARCH for the quadratic assignment problem) An example of the application of the COSEARCH approach has been developed in Talbi and Bachelet (2006) to solve the quadratic assignment problem (QAP). A parallel tabu search is used to solve the QAP, while a genetic algorithm makes a diversification task, which is formulated as another optimization problem. The frequency memory stores information relative to all the solutions visited during the tabu search. The genetic algorithm refers to the frequency memory to generate solutions being in unexplored regions.

Another approach of specialist hybrid HRH heuristics is to use a heuristic to optimize another heuristic (i.e. find the optimal values of the parameters of the heuristic) (Fig. 12). This approach is known as *meta-optimization*. For instance, it has been used to optimize simulated annealing and noisy methods (NM) by GA (Krueger 1993), ant colonies (AC) by GA (Abbattista et al. 1995), simulated annealing based algorithms by a GA (Golden et al. 1998), and a GA by a GA (Shahookar and Mazumder 1990).[7] In Shahookar and Mazumder (1990), the three parameters optimized are the crossover rate, the inversion rate, and the mutation rate. The individuals of the population associated with the optimizer consist of three integers representing the mutation rate, the

[7] This procedure is also called meta-evolution.

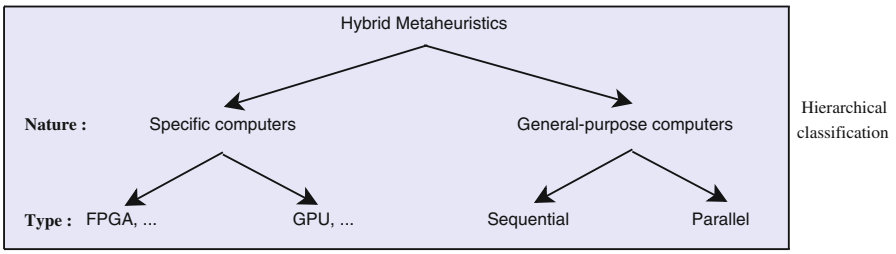


Fig. 13 Classification of hybrid metaheuristics (implementation issues)

inversion rate, and the crossover rate. The fitness of an individual is taken to be the fitness of the best solution that the GA can find in the entire run, using these parameters.

2.2 Implementation issues

The structure of the taxonomy concerning the implementation issues is shown in Fig. 13. A discussion about this taxonomy then follows.

2.2.1 Dedicated versus general-purpose computers

Dedicated computers differ from general purpose computers ones in that they usually only solve a small range of problems, but often at much higher speedups and lower cost. Their internal structure is tailored for a particular problem, and thus they can achieve much higher efficiency and hardware utilization than processors which must handle a wide range of tasks.

In the last years, the advent of programmable logic devices has made easier to build specific computers for metaheuristics such as simulated annealing (Abramson 1992) and genetic algorithms (Salami et al. 1996). A general architecture acting as a template for designing a number of specific machines for different metaheuristics (SA, TS, etc) may be constructed (Abramson et al. 1997). The processor is built with XILINX FPGAs and APTIX interconnection chips. Experiments evaluating a simulated annealing algorithm to solve the traveling salesman problem achieved a speedup of about 37 times over an IBM RS6000 workstation. To our knowledge, this approach has not been yet proposed for hybrid metaheuristics.

Nowadays, the use of GPU (Graphical Processing Unit) devices is more and more popular in many application domains. Indeed, such devices are integrated into many workstations to deal with visualization tasks. The idea is to exploit those available resources to improve the effectiveness of hybrid metaheuristics.

2.2.2 Sequential versus parallel

Most of the proposed hybrid metaheuristics are sequential programs. According to the size of problems, parallel implementations of hybrid algorithms have been considered. The ease of use of a parallel and distributed architecture has been acknowledged for the HTH hybrid model.

2.3 A grammar for extended hybridization schemes

Given a set of metaheuristics A_i, a classification of basic hybridizations has been presented, in which the following notations can be described:

- $LRH(A_1(A_2))$ (homogeneous, heterogeneous) (partial, global) (specialist, general): the metaheuristic A_2 is embedded into the single-solution metaheuristic A_1.
- $HRH(A_1 + A_2)$ (homogeneous, heterogeneous) (partial, global) (specialist, general): the self-contained metaheuristics A_1 and A_2 are executed in sequence.
- $LTH(A_1(A_2))$ (homogeneous, heterogeneous) (partial, global) (specialist, general): the metaheuristic A_2 is embedded into the population-based metaheuristic A_1.
- $HTH(A_1, A_2)$ (homogeneous, heterogeneous) (partial, global) (specialist, general): the self-contained metaheuristics A_1 and A_2 are executed in parallel and cooperate.

These hybridizations should be regarded as primitives that can be combined in different ways. The grammar given in Fig. 14 generalizes the basic hybridization schemes. A practically important property of the grammar is to specify the hybrid heuristic to use, if a metaheuristic problem solving tool is used.

Example 9 (Extended hybridization schemes) Let us present some examples of extended hybridization schemes (Fig. 15). Boese et al. (1994) suggested an adaptive multi-start (AMS) approach, which may be seen as a HRH(LS + LTH(GA(LS))) scheme. First, AMS generates a set of random starting solutions and runs an LS algorithm for each solution to find corresponding local optima. Then, AMS generates new starting solutions by combining features of the T best local optima seen so far, with T being a parameter of the approach. This mechanism bears some resemblance to GAs,

<hybrid-metaheuristic> ⟶ <design-issues><implementation-issue>
<design-issues> ⟶ <hierarchical><flat>
<hierarchical> ⟶ <LRH> | <LTH> | <HRH> | <HTH>
<LRH> ⟶ LRH(<S-metaheuristic>(<metaheuristic>))
<LTH> ⟶ LTH(<P-metaheuristic>(<metaheuristic>))
<HRH> ⟶ HRH(<metaheuristic> + <metaheuristic>)
<HTH> ⟶ HTH(<metaheuristic>)
<HTH> ⟶ HTH(<metaheuristic>, <metaheuristic>)
<flat> ⟶ (<type> , <domain> , <function>)
<type> ⟶ homogeneous | heterogeneous
<domain> ⟶ global | partial
<function> ⟶ general | specialist
<implementation-issue> ⟶ <*specific computers*> | <*general-purpose computers*>
<*specific computers*> ⟶ FPGA | GPU | ...
<*general-purpose computers*> ⟶ sequential | parallel
< *metaheuristic* > ⟶ <*S-metaheuristic*> | <*P-metaheuristic*>
<*S-metaheuristic*> ⟶ LS | TS | SA | TA | NM | GDA | ILS | GRASP | ...
<*P-metaheuristic*> ⟶ EA | SS | ACO | PSO | AIS | BC | EDA | CA | CEA | ...
<metaheuristic> ⟶ <hybrid-metaheuristic>

Fig. 14 A grammar for extended hybridization schemes

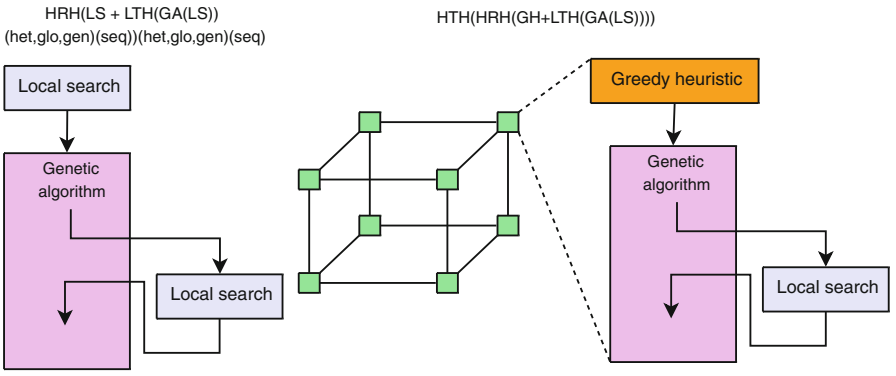


Fig. 15 Extended hybridization schemes

but differs in that many solutions (instead of just two) are used to generate the new starting solutions. New local optima are obtained by running the LS algorithm from these new starting solutions, and the process iterates until some stop criterion is met.

In his PhD thesis, Levine (1994) has used a HTH(HRH(GH+LTH(GA(LS)))) hierarchical scheme to solve set partitioning problems. Efficient results have been obtained with a parallel static implementation in solving big sized problems in real world applications (airline crew scheduling). At the first level, a HTH hybrid based on the island model of parallel genetic algorithms is used. The initial population of each GA is generated by a greedy heuristic (the Chvatal heuristic Chvatal 1979), and a local search algorithm is used to improve the solutions at each generation of the GA. The same hybrid scheme with a sequential implementation has been used in Braun (1990) to solve the traveling salesman problem. The local search algorithms used are the well known 2-opt and or-opt heuristics. The author reported some interesting results on the 442 and 666-city problems. He found the optimum of the 442-city problem, and a solution within 0.04 % of the optimum for the 666-city problem.

The objective of this paper is far from providing an exhaustive list of research works using hybrid metaheuristics. Following this grammar, more than 125 annotated hybrid metaheuristics may be found in Talbi (2002). This shows the usefulness of the taxonomy.

3 Combining metaheuristics with mathematical programming

Metaheuristics and exact algorithms are complementary optimization strategies in terms of the quality of solutions and the search time used to find them. In the last few years, solving exactly important optimization problems using for example integer programming techniques has improved dramatically. Moreover, the availability of efficient optimization software, libraries and frameworks for mathematical programming and high-level modeling languages will lead to more hybrid approaches combining metaheuristics and exact optimization algorithms. In this section, an instantiation and

extension of our classification to hybrid schemes combining mathematical programming approaches and metaheuristics is presented.

3.1 Mathematical programming approaches

The main mathematical programming approaches may be classified as follows:

- **Enumerative algorithms:** This class of algorithms contains tree search algorithms such as branch and bound and dynamic programming. They are based on a divide and conquer strategy to partition the solution space into subproblems and then optimizing individually each subproblem.
- **Relaxation and decomposition methods:** This class of methods are based on relaxation techniques such as the Lagrangian relaxation (Fisher 1985), and decomposition methods such as the Bender's decomposition and the continuous semidefinite programming.
- **Cutting plane and pricing algorithms:** This class of algorithms is based on polyhedral combinatorics in which the search space is pruned.

3.1.1 Enumerative algorithms

Enumerative methods include branch and bound, dynamic programming, A*, and other tree search algorithms. The search is carried out over the whole search space, and the problem is solved by subdividing it in simpler subproblems.

Branch and bound is one of the most popular methods to solve optimization problems in an exact manner. The algorithm is based on an implicit enumeration of all solutions of the considered optimization problem. The search space is explored by dynamically building a tree whose root node represents the problem being solved and its whole associated search space. The leaf nodes are the potential solutions and the internal nodes are subproblems of the total solution space. The size of the subproblems is increasingly reduced as one approaches the leaves.

The construction of such a tree and its exploration are performed using two main operators: *branching* and *pruning* (Fig. 16). The algorithm proceeds in several iterations during which the best found solution is progressively improved. The generated and not yet treated nodes are kept in a list whose initial content is limited to only

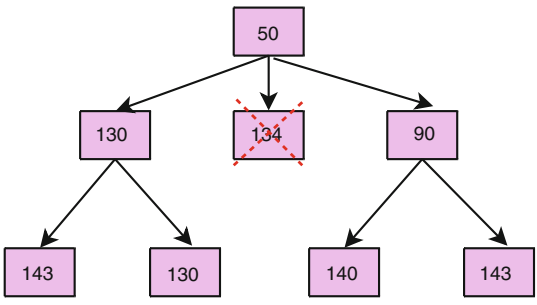


Fig. 16 The branch and bound algorithm. This figure shows the nodes actually explored in the example problem, assuming a depth-first and left-to-right search strategy. The subtree rooted at the second node on level 2 is pruned because the cost of this node (134) is greater than that of the cheapest solution already found (130)

the root node. The two operators intervene at each iteration of the algorithm. The branching strategy determines the order in which the branches are explored. Many branching strategies may be applied such as the *depth-first*, the *breadth-first*, and the *best-first* strategies. The *pruning* strategy eliminates the partial solutions that do not lead to optimal solutions. This is done by computing a lower bound[8] associated to a partial solution. If the lower bound of a node (partial solution) is greater than the best solution found so far or a known upper bound of the problem, the exploration of the node is not needed. The algorithm terminates if there are no more nodes to branch or all nodes have been eliminated. Hence, the most important concepts in designing an efficient branch and bound algorithm are the quality of the bounds and the branching strategy.

The *dynamic programming* (DP) approach is based on the recursive division of a problem into simpler subproblems. This procedure is based on the *Bellman's principle* which says that "the sub-policy of an optimal policy is itself optimal with regard to the start and end states" (Bellman 1957).

Designing a dynamic programming procedure for a given problem needs the definition of the following components (Bertsekas 1998):

- Define the *stages* and the *states*. A problem can be divided into a number of stages N. A number of states are associated to each stage.
- Define the cost of the initial stage and states. There is an initial state of the system x_0.
- Define the recursive relation for a state at stage k in terms of states of previous stages. Let x_k denote the state at stage k. The state of the system changes from x_k to x_{k+1} using the following equation

$$x_{k+1} = f_k(x_k, u_k)$$

  where u_k is a control variable that takes values from a given finite set, which may depend on stage k. The transition from state k to $k+1$ involves a cost $g_k(x_k, u_k)$. The final transition from $N-1$ to N involves the terminal cost $G(x_N)$. The functions f_k, g_k and G must be determined.

Given a control sequence $(u_1, u_2, \ldots, u_{N-1})$, the corresponding state sequence will be $(x_0, \ldots, x_N)$ which is determined from the initial state x_0 using the equation below. In dynamic programming, the objective is to find the optimal control sequence minimizing the total cost:

$$G(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)$$

DP has been successfully applied to knapsack, planning and routing-type problems, in which it is easy to define efficient recursive relationships between stages.

[8] We consider here a minimization problem.

3.1.2 Relaxation and decomposition methods

Relaxation methods consist in relaxing a strict requirement in the target optimization problem. In general, a given strict requirement is simply dropped completely or substituted by another one which is more easily satisfied. The most used relaxation techniques are the LP-relaxation and the Lagrangian relaxation. In addition to their use on solving optimization problems, relaxation methods are also used to generate bounds.

Linear programming relaxation: Linear programming relaxation (LP-relaxation) is a straightforward approach which consists in ignoring the integrality constraints of an integer program (IP). Once the integrality constraints are dropped, the problem can be solved using linear programming solvers. This gives a lower bound for the problem. If the solution found satisfies the integrality constraints (generally not true), it will be the optimal solution for the IP program. If the relaxed problem is infeasible, then so is the IP program. LP-relaxation is widely used in branch and bound algorithms to solve IP problems in which the branching is performed over the fractional variables.

Lagrangian relaxation: Lagrangian relaxations are widely used to generate a problem that is easier to solve. The main idea is to remove some constraints and incorporate them in the objective function. For each constraint, a penalty function is associated. The choice of which constraints are handled in the objective function is important. More complicated constraints to satisfy are preferable as they generate an easiest problem to solve. Given the following LP problem:

$$\begin{aligned} &\text{Max } c^T x \\ &\text{s.t. } Ax \leq b \\ &\text{with } x \in \mathbb{R}^n \text{ and } A \in \mathbb{R}^{m,n} \end{aligned}$$

The set of constraints A is split into two sets: $A_1 \in R^{m_1,n}$ and $A_2 \in R^{m_2,n}$, where $m_1 + m_2 = m$. Then, the subset of constraints A_2 is integrated into the objective function which gives the following Lagrangian relaxation of the original problem:

$$\begin{aligned} &\text{Max } c^T x + \lambda^T (b_2 - A_2 x) \\ &\text{s.t. } A_1 x \leq b_1 \\ &\text{with } x \in \mathbb{R}^n,\ A_1 \in \mathbb{R}^{m_1,n} \text{ and } A_2 \in \mathbb{R}^{m_2,n} \end{aligned}$$

where $\lambda = (\lambda_1, \ldots, \lambda_{m_2})$ are non negative weights which penalize the violated constraints A_2. The efficiency of Lagrangian relaxation depends on the structure of the problem; there is no general theory applicable to all problems. Lagrangian relaxation may find bounds which are tighter than the LP-relaxation. The problem is solved iteratively until optimal (or good) values for the multipliers are found. One of the main issues in the Lagrangian relaxation is the generation of the optimal multipliers. This difficult problem, traditionally attacked through subgradient optimization, can also be solved by metaheuristics.

In practice, decomposition methods are used to solve large IP problems. Among the numerous decomposition approaches one can mention the Bender's decomposition and the Dantzig-Wolfe decomposition.

Bender's decomposition: The Bender's decomposition algorithm is based on the notion of complicating variables. It consists in fixing the values of complicating variables and solves the resulting reduced problem iteratively (Benders 1962). Given a MIP problem:

$$\begin{aligned} &\text{Max } c^T x + h^T y \\ &\text{s.t. } Ax + Gy \leq b \\ &\text{with } x \in \mathbb{Z}_+^n \text{ and } y \in \mathbb{R}_+^p \end{aligned}$$

If the set of variables x is fixed, the following linear program is obtained

$$z_{LP}(x) = max\{hy / Gy \leq b - Ax\}$$

and its dual

$$min\left\{u(b - Ax)/uG \geq h, u \in R_+^m\right\}$$

If the dual polyhedron is assumed to be not empty and bounded, the MIP model can be formulated as follows:

$$z = max_{x \in Z_+^n}(cx + min_{i \in 1,\ldots,T}(u^i(b - Ax)))$$

This model can be reformulated as:

$$z = max\left\{\eta / \eta \leq u^i(b - Ax), i \in 1, \ldots, T, x \in Z_+^n\right\}$$

Then, the algorithm finds cutting planes based on the dual problem. The cutting planes are added to the problem and the problem is re-solved.

3.1.3 Branch and cut and price algorithms

The objective of the following popular techniques is to generate tighter IP relaxations.

Cutting plane: Cutting plane approaches have been proposed in 1958 by Gomory (1958). The use of cuts can improve greatly branch and bound algorithms. In general, cutting plane algorithms consist in iteratively adding some specific constraints to the LP-relaxation of the problem. Those constraints represent restrictions to the problem so that the linear programming polytope closely approximates the polyhedron represented by the convex hull of all feasible solutions of the original IP problem. A good survey of branch and cut algorithms and their use for different families of optimization problems may be found in Juenger et al. (1995), Nemhauser et al. (1999).

Column generation: Column generation has been first applied by Gilmore and Gomory (1961). Column generation generates a decomposition of the problem into

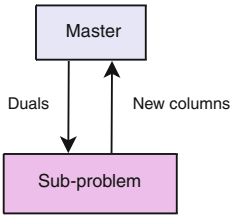


Fig. 17 Branch and price approach

a master and subproblems (Fig. 17). A good survey may be found in Barnhart et al. (1998).

3.2 Classical hybrid approaches

Exact MP algorithms are known to be time and/or memory consuming. In general they cannot be applied to large instances of difficult optimization problems. On one hand their combination with metaheuristics may improve the effectiveness of heuristic search methods (i.e. getting better solutions). On the other hand, this type of combination allows the design of more efficient exact methods (i.e. finding optimal solutions in shorter time). The following sections illustrate, for each class of hybrids belonging to the presented taxonomy, some hybridization schemes combining exact MP algorithms and metaheuristics.

3.2.1 Low-level relay hybrids (LRH)

This class of algorithms represents hybrid schemes in which a metaheuristic approach (resp. exact approach) is embedded into an exact approach (resp. S-metaheuristic approach) to improve the search strategy. In this usual combination, a given metaheuristic or exact algorithm solves a problem of a different nature with respect to the considered optimization problem.

Embedding S-metaheuristics into exact algorithms: Metaheuristics may solve many search problems involved in the design of an exact method such as the node selection strategy, upper bound generation, column generation (Fig. 18):

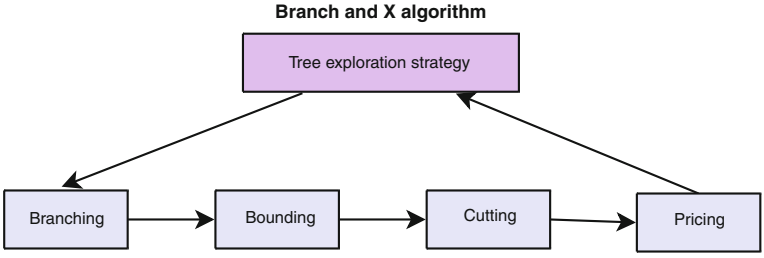


Fig. 18 LRH cooperation in which a metaheuristic can be used in the design of some search components of branch and X family of algorithms (e.g. branch and bound, branch and cut, branch and price): selection of the node to explore, generation of an upper bound, cutting plane generation, column generation selection, etc.)

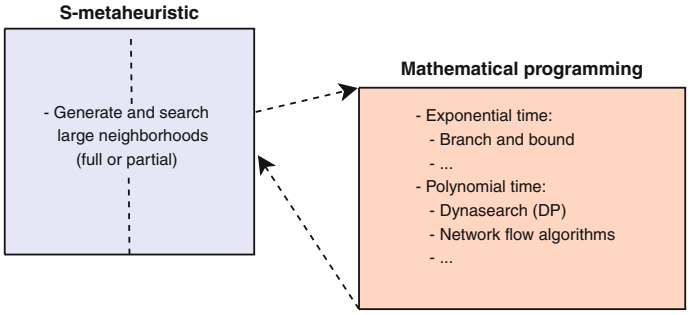


Fig. 19 LRH cooperation where a mathematical programming approach can be used for the efficient search of a very large neighborhood into S-metaheuristics

- **Bounding:** Metaheuristics can provide upper bounds associated to a node of the branch and bound algorithm. Indeed, the partial solution is completed by a given metaheuristic and then a local upper bound is provided.
- **Cutting:** In the branch and cut algorithm, the cutting plane generation problem is a crucial part of the algorithm: the part that looks for valid inequalities that cut off the current non-feasible linear program (LP) solution. Metaheuristics may be used in this *separation procedure*. For instance, this approach has been proposed for the CVRP (Capacitated Vehicle Routing Problem) (Augerat et al. 1998). Some metaheuristics (e.g. tabu search, greedy heuristics) have been designed to extract a set of violated capacity constraints of the relaxed problem.
- **Pricing:** In the branch and price algorithm, the pricing of columns may be carried out by a metaheuristic (Filho and Lorena 2000).

Some metaheuristic ingredients may also be used in tree search algorithms such as the concepts of tabu lists and aspiration criteria (Prestwich 2002).

Embedding exact algorithms into S-metaheuristics: Many combinations may be designed in which exact algorithms are embedded into search components of S-metaheuristics.

Very large neighborhoods: S-metaheuristics may be improved using very large neighborhoods. The concept of large neighborhoods is also used in the ILS (perturbation operator Lourenco 1995) and the VNS metaheuristics. Mathematical programming approaches may be used to search efficiently large neighborhoods to find the best or an improving solution (Fig. 19). Algorithms such as branch and bound, dynamic programming (Potts and Velde 1995), network flow algorithms (Dowsland 1998), and matching algorithms (Gutin 1999) have been used to explore large neighborhoods defined for different important optimization problems. Some hybrid schemes explore the *whole* neighborhood while other neighborhood search algorithms explore a *subset* of the neighborhood. If no polynomial time algorithm exists to search the whole neighborhood, a partial search is generally performed.

Example 10 (Hyperopt) The hyperopt S-metaheuristic explores only a subset of a very large neighborhood (Burke et al. 2001). The hybrid algorithm has been used to solve the asymmetric traveling salesman problem. The move operator is based

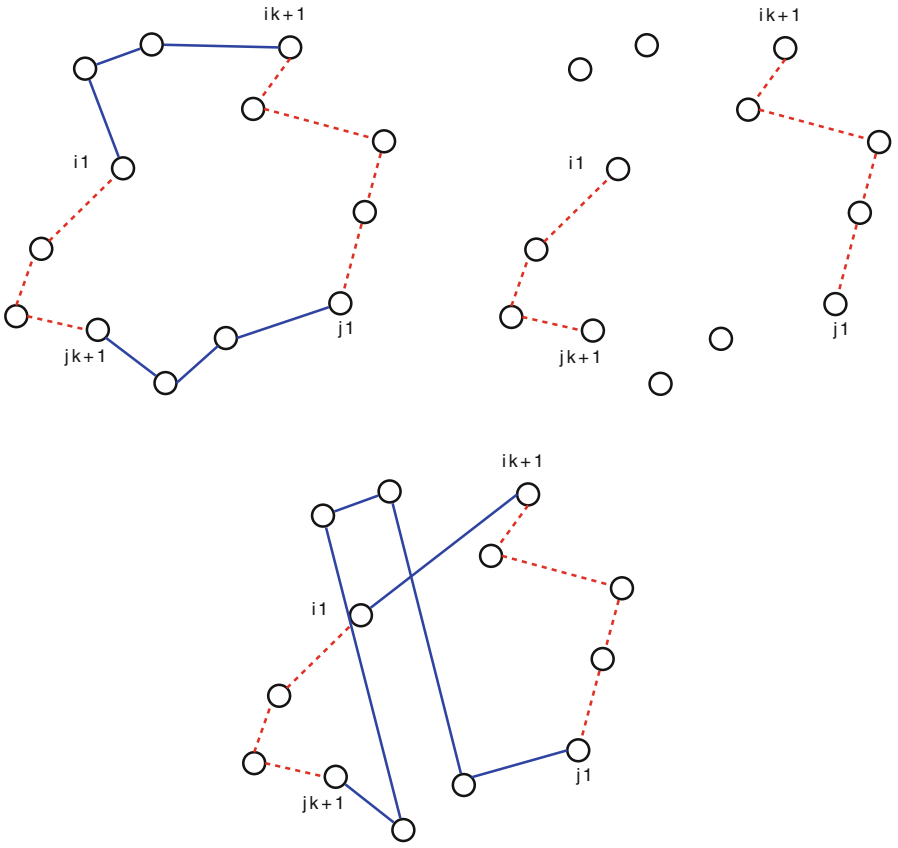


Fig. 20 Large neighborhood based on hyperedge for the asymmetric traveling salesmen problem

on hyperedges which represent subpaths of the tour. A hyperedge $H(i, j)$ is represented by its start node i, end node j and length k. A k-hyperopt move consists in deleting two hyperedges $H(i_1, i_{k+1})$ and $H(j_1, j_{k+1})$ of length k. It is supposed that $H(i_1, i_{k+1}) \bigcap H(j_1, j_{k+1}) = \phi$, (i.e. the hyperedges share no common edges). Then, the move operator adds edges to the hyperedges $H(i_{k+1}, j_1)$ and $H(j_{k+1}, i_1)$ to construct a feasible tour (Fig. 20). The size of the hyperedge neighborhood grows exponentially with k. The neighborhood search algorithm is reduced to a smaller TSP problem. The algorithms used are based on enumeration for small k and a dynamic programming algorithm for medium values of k.

Some search concepts of exact algorithms may be used in S-metaheuristics. Efficient mathematical programming approaches that generate "good" lower bounds exist for many optimization problems. Information related to lower bounds, obtained for example by Lagrangian relaxation, can be exploited into a metaheuristic to intensify the search in promising regions of the search space.

For instance, information based on Lagrangian multipliers are exploited to guide the metaheuristic in solving the set covering problem (Beasley 1990). Lower bounds

have been used in S-metaheuristics such as tabu search to improve the search (Hindi et al. 2003; Dowsland 1998).

3.2.2 Low-level teamwork hybrids (LTH)

Recall that in this class of hybrid algorithms, a search component of a P-metaheuristic is replaced by another optimization algorithm. Concerning the combination of P-metaheuristics and MP algorithms, two main hybrid approaches may be considered: exact search hybrid algorithms in which a P-metaheuristic is embedded into an exact algorithm, and heuristic search algorithms in which an exact algorithm is embedded into a P-metaheuristic.

Embedding a P-metaheuristic into an exact algorithm: The questions arising in designing a branch and bound algorithm are:

- **Branch ordering:** How the problem to solve (node of the tree) is decomposed into subproblems? On which variable the next branching is applied? Indeed, near-optimal solutions obtained by metaheuristics may guide the branch and bound to apply an efficient branch ordering by giving preference to branches which share common values with near-optimal solutions.
  The node selection problem in tree search based on branch and bound may be solved by metaheuristics. For instance, genetic programming approaches have been used to deal with the node selection problem (Kostikas and Fragakis 2004).
- **Variable selection:** In which subproblem (child node) the search will be performed in the next step? What value should first be assigned to the branching variable? Information obtained from branch and bound tree can be used by heuristic algorithms to determine a better strategy for variable selection (Kostikas and Fragakis 2004).

An exact algorithm constructs partial solutions which are used to define a search space for a metaheuristic. Then, the obtained results are exploited in order to refine the bounds or generate the columns into a branch and cut algorithm.

Example 11 (Local branching) The *local branching* exact approach has been proposed in Fischetti and Lodi (2003). It uses the principle of local search heuristics. The search space is partitioned by introducing branching conditions expressed through (invalid) linear inequalities called local branching cuts. Let us consider a MIP problem with $\{0, 1\}$ variables. The $k - opt$ neighborhood is considered. The main principle of the local branching method is to iteratively solve a subproblem corresponding to the neighborhood $k - opt$ of a partial solution s. Two partitions are then considered: $p_1 = \{x \in \{0, 1\}^n / \Delta(x, s) \leq k\}$ and $p_2 = \{x \in \{0, 1\}^n / \Delta(x, s) \geq k + 1\}$, where Δ represents the Hamming distance, and n the size of the problem. The problem associated to p_1 is solved. A new subproblem is generated if an improved solution is found. Otherwise, the other problem is solved using the standard procedure.

Embedding an exact algorithm into P-metaheuristic: In this hybrid scheme, some search components of a P-metaheuristic induce optimization problems which are solved by exact algorithms (Fig. 22).

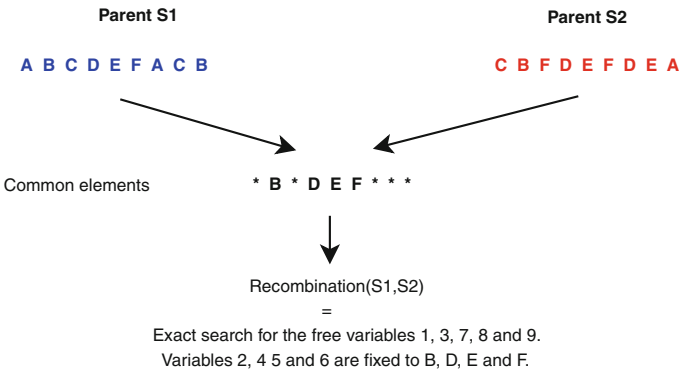


Fig. 21 Using exact algorithms into recombination operators (e.g. crossover) of P-metaheuristics

Example 12 (Exact algorithms into recombination operators) Exact algorithms may be integrated into recombination operators of P-metaheuristics such as evolutionary algorithms to find the best offspring from a large set of possibilities. The induced problem *Recombin-ation*(S_1, S_2) is defined to generate the best offsprings from the parents S_1 and S_2. A common idea is to keep the common elements of the parents and explore all the other possibilities to generate better offsprings (Fig. 21).

For instance, a branch and bound algorithm (resp. dynamic programming) has been used into the crossover operator of a genetic algorithm in solving permutation problems (Crainic and Toulouse 2003) (resp. Yagiura and Ibaraki 1996).

For some specific problems, polynomial exact algorithms may also be used such as minimum spanning tree algorithms (Jahuira and Cuadros-Vargas 2003), matching algorithms in a bipartite graph (Aggarwal et al. 1997; Balas and Niehaus 1998) for optimized crossover operators.

Large neighborhood search algorithms integrated in P-metaheuristics typically belong typically to the LTH class of hybrids. For instance, the mutation operator in EAs can also be substituted by MP algorithms which explore large neighborhoods (Fig. 22).

Exact decoding: Exact algorithms can also be used as decoders of incomplete solutions carried out by metaheuristics. This hybrid strategy is applied in the case where

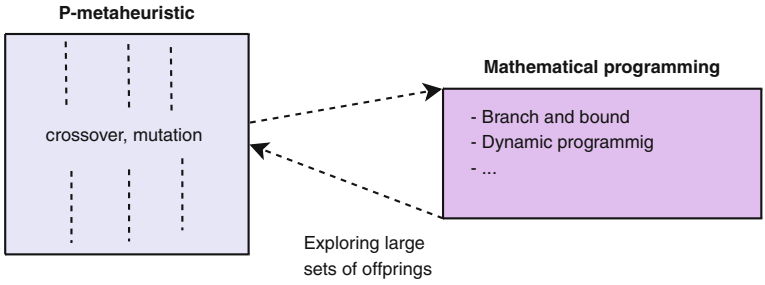


Fig. 22 LTH heuristic cooperation: exact algorithms are used as search components of a P-metaheuristic (e.g. recombination, mutation)

metaheuristics use an incomplete encoding for the problem. Once a good incomplete solution is found, exact algorithms optimally complete the missing part of the encoding.

Exact search ingredients: Some search ingredients of exact algorithms can also be used in P-metaheuristics:

- **Lower bounds:** The use of lower bounds into a P-metaheuristic can improve the search. Lower bounds have been used in the construction phase of the ant colonies P-metaheuristic to solve the quadratic assignment problem (QAP) (Maniezzo 1999). The well known Gilmore-Lawler lower bound and the values of the dual variables are used to order the locations during the construction phase. The impact of the location in a given QAP instance depends on the value of its associated dual variable. The concept of bounds has been used into evolutionary algorithms for the mutation and the crossover operators (Tamura et al. 1994; Dowsland et al. 2006). Indeed, partial solutions that exceed a given bound are deleted. The bounds are computed using the linear and Lagrangian relaxation, and tree search methods.
- **Partial solutions:** The evaluated partial solutions (subproblems) maintained by the branch and bound family of algorithms may provide interesting initial solutions to improve. The evaluation of those partial solutions will guide the metaheuristics to more promising regions of the search space (Maniezzo 1999). The partial solution with the least cost lower bound suggests a promising region by giving additional information.

3.2.3 High-level relay hybrids (HRH)

This class of cooperation, where self-contained algorithms are used in sequence, is the most popular in practice. This may be seen as a pre-processing or a post-processing step. Some information is provided between the two families of algorithms (metaheuristics and MP algorithms) in a sequential way (Fig. 23).

Information provided by metaheuristics: In the case where the information is provided by the metaheuristics, the most natural and trivial hybrid approach is to start with a metaheuristic to find a "good" *upper bound* which will be used by a MP algorithm

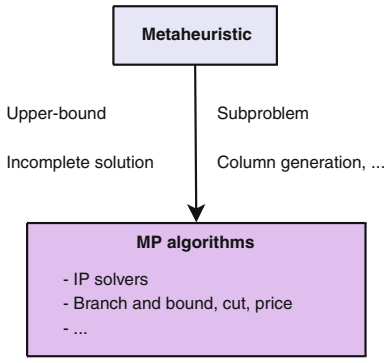


Fig. 23 HRH cooperation: information provided by metaheuristics to MP algorithms

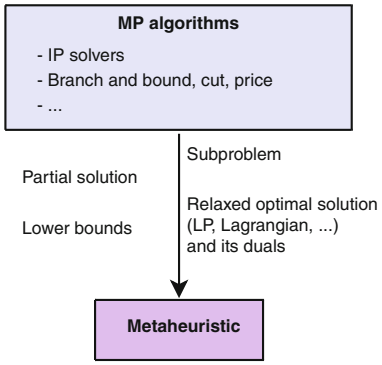


Fig. 24 HRH cooperation: information provided by MP algorithms to metaheuristics

in the bounding phase (Fig. 24). Indeed, the efficiency of the search (pruning phase) is largely dependent on the quality of the upper bound.

Using the characteristics of generated high quality solutions, metaheuristics can be used to reduce the size of the original problem. Then, the exact method can be applied to solve the reduced problem. This approach is interesting for optimization problems where "good solutions" share many components (Applegate and Cook 1991). This allows the reduction of the problem into much smaller problems which can be solved exactly by state-of-the-art mathematical programming algorithms. The reduction phase may concern:

– **Partitioning of decision variables:** In this strategy, the decision variables are partitioned into two sets X and Y. The metaheuristic will fix the variables of the set X and the exact method will optimize the problem over the set Y. Hence, the generated subproblems are subject to free variables in the set Y and fixed variables in the set X. Those subproblems are solved exactly.
A set of high quality solutions may be obtained by a P-metaheuristic or an iterated S-metaheuristic. The characteristics of this set can be exploited to define smaller problems by fixing some variables and solving the resulting subproblems by exact algorithms. An example of such strategy is the Mimausa method for the quadratic assignment problem (Mautor and Michelon 1997). The method builds at each iteration a subproblem by fixing k decision variables and solves it by a branch and bound algorithm.

Example 13 (Reducing problems by metaheuristics to be solved by MP algorithms) Analyzing the landscape for the TSP problem, one can observe that local optimum solutions share many edges with the global optimum and they are concentrated in the same region of the search space (big valley structure) (Talbi et al. 1998). This characteristic has been exploited in Cook and Seymour (2003) to design one of the most efficient heuristic for the TSP: the tour merging heuristic. The tour merging heuristic consists of two phases: the first phase generates a set T of "good" tours using the Lin-Kernigham algorithm on the input graph $G = (V, E)$. Then, a dynamic programming algorithm is applied on a restricted graph $G' = (V, E')$, where $E' = \{e \in E / \exists t \in T, e \in t\}$. The exact algorithm solves instances up to 5,000 cities.

For the p-median problem, the same remark holds in the analysis of its landscape (Rosing and ReVelle 1997). The first phase is based on an iterated S-metaheuristic using different random initial solutions. The problem is reduced in terms of the number of nodes (location facilities) using the *concentration set* (CS) concept. The integer programming model of the restricted problem is solved using respectively a linear programming relaxation (CPLEX solver) and a branch and bound. The authors exploit the fact that more than 95 % of linear programming relaxation optimal solutions are integers.

For continuous optimization, HRH hybrid schemes are very popular. For instance, a hybrid method combining tabu search and the simplex algorithm provides interesting results in solving complex continuous functions (Chelouah and Siarry 2004).

– **Domain reduction:** This strategy consists in reducing the domain of values that the decision variables can take. The metaheuristic will perform a domain reduction for the decision variables and then an exact method is used over the reduced domains. For instance, a GA may be used to find promising ranges for decision variables and then tree search algorithms are considered to find the optimal solution within those ranges (Nagar et al. 1995).

Information provided by exact algorithms: In the case where the information is provided by an exact algorithm, many hybrid approaches may be designed:

– **Partial solutions:** Partial solutions are first provided by an exact algorithm which are then completed by a metaheuristic.
– **Problem reduction:** In this strategy, a problem reduction is carried out by an exact algorithm. For instance, a tree search algorithm has been used to reduce the size of a nurse scheduling problem (Dowsland and Thomson 2000). Then, a tabu search strategy is applied to solve the problem within a simplified objective function (Dowsland and Thomson 2000).
– **Relaxed optimal solutions and their duals:** The optimal solutions for relaxed formulations (e.g. LP-relaxation, Lagrangian relaxation) of the problem and its duals may be exploited by metaheuristics.

Example 14 (LP-relaxations as an input for metaheuristics) Information gathered from solutions obtained by LP-relaxations of MIP problems may be used as an input for a metaheuristic. A straightforward approach is the "dive and fix" strategy, where the value of a subset of the integer variables are fixed and the resulting LP problem is solved. This strategy iterates until the LP finds an integer solution. This will restrict the search space of metaheuristics in promising regions. This idea has been used to design an efficient hybrid approach for the 0–1 multidimensional knapsack problem (Vasquez and Hao 2001). Many linear relaxations of the MIP formulation of the problem including different constraints on the number of elements of the knapsack are solved exactly. The obtained solutions are exploited to generate initial solutions for multiple tabu search metaheuristics.

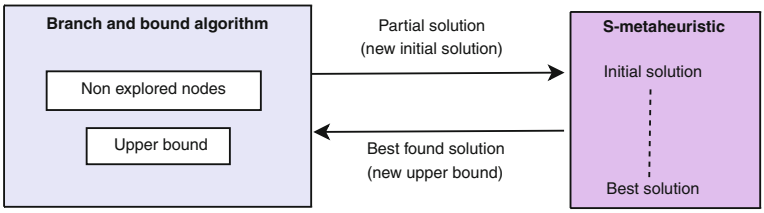


Fig. 25 HTH cooperation between metaheuristics and MP algorithms

3.2.4 High-level teamwork hybrids (HTH)

Few strategies belong to this class of hybrids which combines metaheuristics and MP algorithms in a parallel cooperative way. However this is a promising class of hybrids. A set of agents representing metaheuristics and MP algorithms are solving global, partial or specialist optimization problems and exchanging useful informations. A majority of proposed approaches fall in the class of partial and specialist hybrids. Indeed, the search space is generally too large to be solved by an exact approach. One of the main issues in the HTH hybrid is the information exchanged between metaheuristics and MP algorithms. The different complementary algorithms solving different problems may exchange any information gathered during the search to improve the efficiency and the effectiveness of the hybrid approach: solution(s), subproblems, relaxed optimal solutions and their duals, upper bounds, lower bounds, optimal solutions for subproblems, partial solutions, etc.

Example 15 (Parallel cooperation between a branch and bound and a S-metaheuristic) In a parallel cooperation between branch and bound algorithms and a S-metaheuristic, the following information may be exchanged (Fig. 25):

- **From a branch and bound to a S-metaheuristic:** A subproblem of the branch and bound (node of the tree, partial solution) with least-cost lower bound may be used by a S-metaheuristic to generate an initial solution. The lower bound is used to predict potential interesting search regions. This process may be initiated as a diversification search, when the classical "intensification" process is terminated. Indeed, this partial solution provides a promising area for a S-metaheuristic to explore. The non explored node list maintained by a branch and bound provides a metaheuristic with new initial solutions.
- **From a S-metaheuristic to a branch and bound algorithm:** The best solution identified so far by a metaheuristic may be used in branch and bound algorithms for a better pruning of the search tree. Indeed, the better the upper bound, the more efficient the pruning of the search tree. This information is exchanged each time the best solution found is improved.

In general and global hybrids, where all the algorithms are solving the same target problem, the space of design is reduced. For instance, a parallel HTH hybrid which consists in combining a branch and bound algorithm with simulated annealing has been designed (Nwana et al. 2005). The SA algorithm sends improved upper bounds

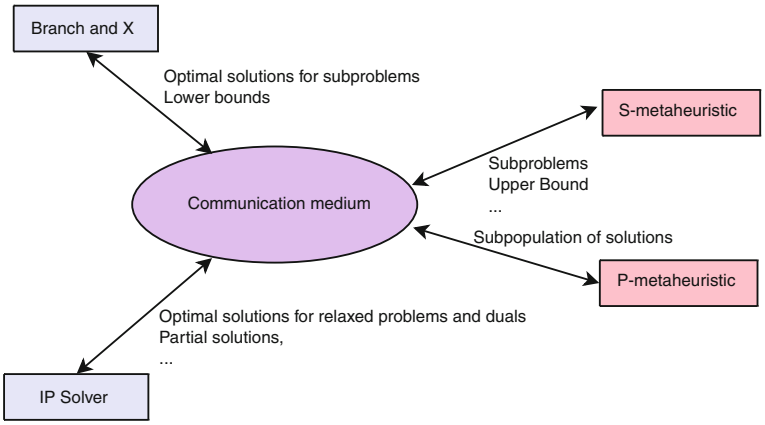


Fig. 26 Specialist HTH cooperation between S-metaheuristics and MP algorithms

to the exact algorithm. Any integer bound obtained by the branch and bound execution is passed to SA and used as an alternative reheated solution.

In specialized hybrids, where the algorithms are solving different problems, many strategies may be proposed (Fig. 26). For instance, a parallel cooperation between a local search metaheuristic and a column generation (branch and price) algorithm to solve the VRP problem has been proposed (Chabrier et al. 2002). The local search algorithm is used to generate new columns for a branch and cut algorithm.

The grammar presented in section 14 has been extended to hybrid schemes combining metaheuristics with exact optimization algorithms (Jourdan et al. 2009). More than 60 annotated hybrid approaches are detailed in the paper. Other examples of combining metaheuristics with exact algorithms may be found in the survey papers (Puchinger and Raidl 2005; Dimitrescu and Stutzle 2003; Jourdan et al. 2009).

4 Combining metaheuristics with constraint programming

Constraint programming (CP) is a modeling and an exact[9] search paradigm based on constraint satisfaction techniques which are largely used in the artificial intelligence community (Apt 2003). CP has been applied successfully to many combinatorial optimization problems with tightly-constrained search problems, while metaheuristics perform well for under-constrained optimization problems.

Nowadays, more and more hybrid approaches combining metaheuristics and constraint programming are used to solve optimization problems. Indeed, metaheuristics and constraint programming are complementary search and modeling approaches, which may be combined naturally to solve optimization problems in a more efficient manner (Focacci et al. 2002). One of the main advantages of using constraint programming is its flexibility. Models are based on a declarative programming paradigm. Hence, the addition/deletion of new constraints in the model is straightforward.

[9] The term complete is always used in the CP community.

4.1 Constraint programming

Feasibility problems in constraint programming are modeled by means of a set of variables linked by a set of constraints. The variables take their values on a finite domain of integers. The constraints may have mathematical or symbolic forms. *Global constraints* refer to a set of variables of the problem. An example of such global constraints is `all_different`$(x_1, x_2, \ldots, x_n)$ which specifies that all the variables $x_1, x_2, \ldots, x_n$ must be different.

Solving a feasibility problem in CP is based on interleaving the *propagation* and the *search* processes in order to find a feasible solution for the problem. Minimizing an objective function may be reduced to solve a given number of feasibility problems.

A propagation algorithm is associated to each (or a set of) constraint(s). It consists in filtering (or reducing) from variable domains the values that cannot lead to feasible solutions. The propagation algorithm is terminated once no more values can be eliminated from the variable domains.

Once the propagation phase is finished, there may remain some inconsistent values in the variable domains. Therefore, a *search* algorithm is launched. The search algorithm is based on a tree search procedure where a branching step is applied by partitioning the current problem into subproblems. Branching may be done by instantiating a given variable to a feasible value of its domain or adding a new constraint.

The questions arising in designing a search algorithm in CP are more or less similar to those of branch and bound algorithms:

- **Branch ordering:** How the problem to solve (node of the tree) is split into subproblems when the propagation algorithm is inconclusive? On which variable the branching is applied next?
- **Variable selection:** From which subproblem (child node) the search continues next? What value should be first assigned to the branching variable?

4.2 Classical hybrid approaches

Many combination schemes show that the hybridization of metaheuristics and CP is fruitful for some optimization problems. The following sections illustrate, for each class of hybrids belonging to the presented taxonomy, some hybridization schemes between constraint programming algorithms and metaheuristics. Some illustrative examples may also be found in Focacci et al. (2002).

4.2.1 Low-level relay hybrids (LRH)

As within mathematical programming approaches, constraint programming may be used to explore large neighborhoods in S-metaheuristics (full or partial). Indeed, when the propagation tends to reduce the search space, CP is an efficient approach in modeling the expression of neighborhoods and exploring very large neighborhoods with side constraints (Shaw 1998). Two different types of exploration may be applied:

- **Neighborhoods with expensive testing of feasibility:** Neighborhoods around the current solution are defined by adding side constraints to the original problem. Checking the feasibility for all side constraints by CP may be efficient. Indeed, the feasibility test of solutions may be an expensive task. The propagation algorithms of CP reduce the size of neighborhoods.
- **Large neighborhoods:** Optimizing the exploration of the neighborhood with inlined constraint checks. For instance, the problem of searching very large neighborhoods is tackled with a constraint programming solver in the resolution of vehicle routing problems (Pesant and Gendreau 1999). A CP model has been proposed for the neighborhood, where every feasible solution represents a neighbor. A given subset of decision variables may also be fixed (Applegate and Cook 1991). A CP search has been carried out over the uninstantiated variables to solve a scheduling problem. A similar approach has been proposed in Shaw (1998) for a vehicle routing problem, and in Caseau and Laburthe (1995) for a job-shop scheduling problem.

4.2.2 Low-level teamwork hybrids (LTH)

In this class of LTH hybrids between metaheuristics and CP, two main categories may be distinguished: exact search hybrid algorithms in which a metaheuristic is embedded into constraint programming, and heuristic search algorithms in which constraint programming is embedded into a P-metaheuristic.

Embedding metaheuristics into constraint programming: Metaheuristics may be used to improve the search algorithm in CP. The following hybrid approaches may be applied to converge more quickly to the optimal solution or approximating "good" solutions:

- **Node improvement:** Metaheuristics may be applied to partial solutions of the tree to improve or repair the nodes of the search tree. A greedy approach may also explore a set of paths from a node of the search tree. Then, CP search continues from the improved solutions (Caseau and Laburthe 1999).
- **Discrepancy-based search algorithms:** This approach generates near-greedy paths in a search tree. This approach has been used in limited discrepancy search (Harvey and Ginsberg 1997) and dynamic backtracking (Ginsberg 1993). Lookahead evaluation of greedy algorithms may also be used over the nodes of the search tree (Caseau and Laburthe 1999).
- **Branch ordering:** Metaheuristics may be applied to answer the following question: which child node to investigate first when diving deeper into the node. Metaheuristics may be used to give a preference to a branch that is consistent with the near-optimal solution. Indeed, the use of metaheuristics produces a better variable ordering and then will speedup the tree search. This approach has been used for solving satisfiability problems (Sellmann and Ansótegui 2006).
  Metaheuristics may also be considered to solve relaxed problem. At each node, the subproblem is relaxed by removing some constraints. The violated constraints in the obtained solution will form the basis for branching. for instance, this approach

has been used to solve scheduling problems (Kamarainen and Sakkout 2002; Narayek et al. 2003).

- **Variable selection:** Metaheuristics are used for variable selection at each node of the tree. This strategy consists in reducing the list of candidates. A straightforward strategy has been used in Feo and Resende (1995). Let $v_1, v_2, \ldots, v_n$ be the possible branches in a decreasing order of preference (lower bound $h(v_i)$). The strategy consists in selecting the v_i branches such as $h(v_i) \leq h(v_1) + \alpha(h(v_n) - h(v_1))$ where $\alpha \in [0, 1]$ is a parameter. Those branches constitute the RCL list (Restricted Candidate Lists), whereas the other branches are not explored.
- **Branching restriction:** Metaheuristics may be used to filter the branches of the tree-search node. This hybrid scheme has been proposed to solve a scheduling problem (Cesta et al. 2001).

CP can construct partial solutions which are used to define a search space for a metaheuristic. Then, the obtained results are used in order to refine the bounds or columns to generate in a branch and cut algorithm.

Embedding constraint programming into P-metaheuristics: Some search components of a P-metaheuristic induce optimization problems which are solved by CP. For instance, some recombination operators such as crossover in EAs may be optimized using CP. In addition to the recombination operators, large neighborhood search algorithms based on CP can be integrated into unary operators of P-metaheuristics such as the mutation in EAs.

Some search ingredients of constraint programming algorithms can also be used into P-metaheuristics. For instance, the use of lower bounds into a P-metaheuristic can improve the search. The partial solutions (subproblems) maintained by CP may provide to metaheuristics interesting initial solutions to metaheuristics. The evaluation of those partial solutions will guide the metaheuristics to more promising regions of the search space. The partial solution with the least cost lower bound suggests a promising region.

CP algorithms can also be used as decoders of indirect representations carried out by metaheuristics. This strategy may be applied once the metaheuristics use indirect encoding which represent incomplete solutions of the problem. This strategy is efficient when the decoding involves complex constraints to satisfy.

4.2.3 High-level relay hybrids (HRH)

In this class of hybrids, self-contained metaheuristics are used in conjunction with CP in a pipeline manner. Metaheuristics are considered as a pre-processing or a post-processing step for CP.

Information provided by metaheuristics: In the case where the informations are provided by metaheuristics, similar information exchanges as with mathematical programming algorithms may be used: upper bounds, incomplete solutions, subproblems, etc.

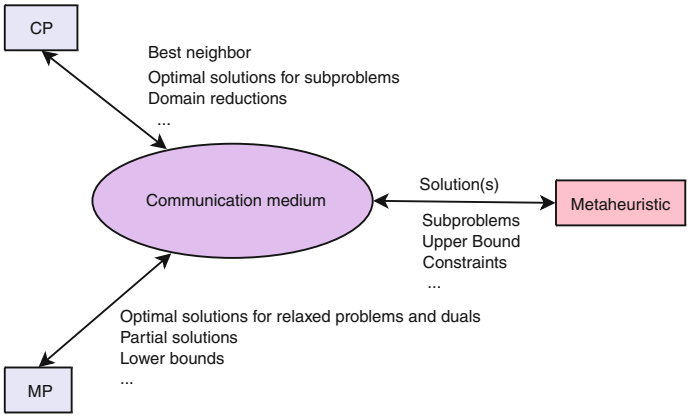


Fig. 27 HTH cooperation between metaheuristics, MP and CP strategies

Information provided by constraint programming: In the case where the information is provided by CP, the same information as with mathematical programming[10] may be considered: partial solutions (i.e. subproblems), optimal solutions for relaxed problems, etc.

For instance, heuristic search-based hybrid scheme may be applied in solving some generated subproblems by CP. A subset of variables are assigned values using a complete search approach. This approach has been used for scheduling problems (Nuijten and Pape 1998) and routing problems (Shaw 1998). This hybrid scheme has been also proposed to solve satisfiability (SAT) problems, where a depth-bounded tree search is carried out and a local search procedure is applied at nodes reaching the depth-limit (Habet et al. 2002). CP can be also applied to an incomplete formulation of the problem. For instance, all (or a set of) feasible solutions are generated by a CP strategy. Then, a metaheuristic will be applied to improve feasible solutions represented by the leaves of the CP tree.

4.2.4 High-level teamwork hybrids (HTH)

Few hybrid HTH strategies combining CP and metaheuristics have been investigated. This class constitutes a promising way to develop efficient solvers and optimization algorithms. The architecture of this class of hybrids may be viewed as a set of agents implementing different strategies (CP, metaheuristics, MP) in solving the target problem, and different subproblems and relaxed problems. Those agents will exchange information on the search. For an exact approach, the objective is to speedup the search in obtaining an optimal solution (efficiency). For a heuristic strategy, the objective is also to improve the quality of the obtained solutions (effectiveness). The exchanged information may include: solution(s), subproblems, relaxed optimal solutions, upper bounds, lower bounds, optimal solutions for subproblems, partial solutions, etc (Fig. 27).

[10] However, the duals cannot be considered.

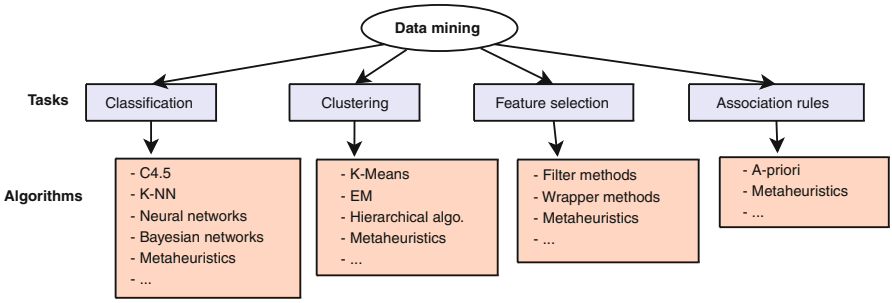


Fig. 28 Some data mining tasks and associated algorithms

5 Hybrid metaheuristics with machine learning and data mining

Combining metaheuristics with data mining and machine learning techniques represents another way to improve the efficiency and effectiveness of the optimization algorithms based on metaheuristics.

5.1 Data mining techniques

Data mining (DM), also known as knowledge discovery in databases (KDD), is the process of automatically exploring large volumes of data (e.g. instances described according to several attributes), to extract interesting knowledge (patterns). In order to achieve this goal, data mining uses computational techniques from statistics, machine learning and pattern recognition.

Various data mining tasks can be used depending on the desired outcome of the model. Usually a distinction is made between supervised and unsupervised learning. Classical tasks of supervised learning are (Fig. 28):

- **Classification:** examining the attributes of a given instance to assign it to a predefined category or class.
- **Classification rule learners:** discovering a set of rules from the data which forms an accurate classifier.

The most common tasks of unsupervised learning are:

- **Clustering:** partitioning the input data set into subsets (clusters), so that data in each subset share common aspects. The partitioning is often indicated by a similarity measure implemented by a distance.
- **Association rule learners:** discovering elements that occur in common within a given data set.

The feature selection task objective consists in reducing the number of attributes (i.e. dimensionality of the data set). Feature selection is often considered as a necessary preprocessing step to analyze data characterized by a large number of attributes. It allows to improve the accuracy of the extracted models. Two models of feature selection exist depending on whether the selection is coupled with a learning scheme or not. The first one, the *filter model*, which carries out the feature subset selection

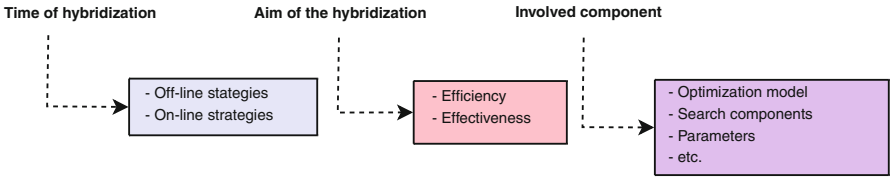


Fig. 29 Some ways integrating knowledge into metaheuristics

and the learning (e.g. classification, clustering) in two separate phases, uses a measure that is simple and fast to compute. The second one, the *wrapper method*, which carries out the feature subset selection and learning in the same process, engages a learning algorithm to measure the accuracy of the extracted model. From the effectiveness point of view, wrapper methods are clearly advantageous, since the features are selected by optimizing the discriminate power of the finally used learning algorithm. However, their drawback is a more important computational cost.

Metaheuristics have been largely used to solve data mining tasks with a great success. However, using data mining techniques to improve the efficiency and effectiveness of metaheuristics is less studied. This hybridization scheme can be viewed as knowledge extraction and integration into metaheuristics. This knowledge may take different forms. Figure 29 describes some ways to integrate knowledge into metaheuristics.

Three criteria will be used to refine our classification (Jourdan et al. 2006):

- **Time of knowledge extraction:** Two kinds of hybridizations can be distinguished depending on the time of extracting the used knowledge. Hybridizations which extract the knowledge before the search starts are called *off-line knowledge* strategies and combinations where the knowledge is extracted dynamically during the search are described as *on-line knowledge* strategies.
- **Aim of the hybridization:** Either the combination allows to improve the efficiency of the algorithm by reducing the search time, or the combination is used to improve the effectiveness of the algorithm leading to better quality of solutions. The efficiency may be carried out by approximating the objective function or reducing the size of the search space. The effectiveness may be improved by incorporating some knowledge into the search components or by updating the parameters of the metaheuristics in an adaptive way. Of course, a given hybridization may improve both criteria: efficiency and effectiveness.
- **Involved component:** A metaheuristic is composed of different search components. Hybridization can occur in any search component such as encoding of solutions, initialization of solutions, search variation operators (e.g. mutation, crossover, neighborhood), etc. It may also be used to fix the parameters of the algorithm or defining the optimization problem to solve (e.g. objective function).

5.2 Main schemes of hybridization

In the following sections, some hybridization schemes between metaheuristics and data mining techniques are presented according to each class of the general taxonomy.

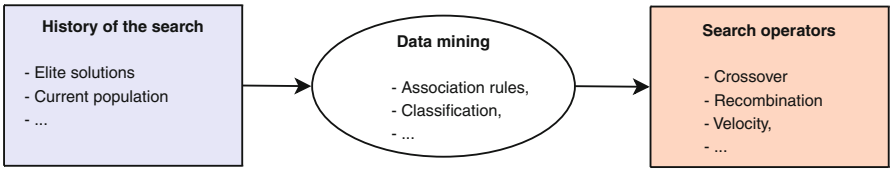


Fig. 30 Extracting knowledge from the history of the search and its use into search operators

5.2.1 Low-level relay hybrid (LRH)

Traditional S-meta-heuristics, greedy or multi-start strategies (e.g. GRASP algorithm) do not use any information on the search of previous iterations to initialize the next search even if the tabu search algorithm uses the concept of memory to guide the search. Hence, some knowledge may be introduced in those families of metaheuristics.

Optimization model: The extracted knowledge may be used to transform the target optimization problem. For instance, in the ART (Adaptive Reasoning Technique) online approach, the search memory is used to learn the behavior of a greedy algorithm (Patterson et al. 1999). Some constraints are added to the problem. Those constraints are generated from the non interesting visited solutions according to the values associated to their decision variables. Similar to the tabu list strategy, those constraints are dropped after a given number of iterations.

Parameters setting: Another LRH hybrid approach provides a dynamic and adaptive setting of the parameters of a S-metaheuristic. Indeed, knowledge extracted during the search may serve to change dynamically at run time the values of some parameters such as the size of the tabu list in tabu search, the temperature in simulated annealing.

This dynamic setting may also concern any search component of a S-metaheuristic such as the neighborhood and the stopping criteria.

5.2.2 Low-level teamwork hybrids (LTH)

This hybrid scheme is very popular in P-metaheuristics.

Search components: A straightforward LTH hybrid approach consists in using data mining techniques in recombination operators of P-metaheuristics. In this class of hybrids, the knowledge extracted during the search is incorporated into the recombination operators for the generation of new solutions (Fig. 30). From a set of solutions (e.g. current population, elite solutions), some models are extracted which may be represented by classification rules, association rules, decision trees, etc. Those models (patterns) will participate in the generation of new solutions to intensify or diversify the search.

Example 16 (Integrating knowledge into recombination operators) In this hybrid scheme, a set of decision rules describing the generated solutions are extracted. For instance, classification rules describing the best and worst individuals of the current population are extracted (Michalski 2000). Those rules are generated using the *AQ learning algorithm*, a general decision rules learning algorithm (Fig. 31). The extracted

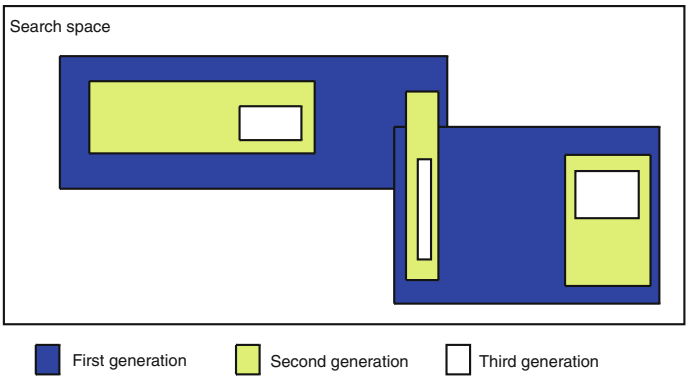


Fig. 31 Reduction of the search space for the offsprings using a learnable evolution model

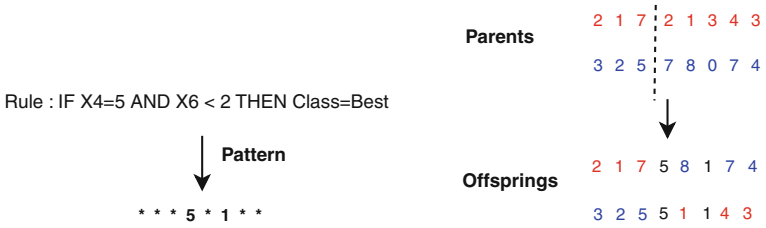


Fig. 32 Crossover operator using the induced rule as a pattern. For instance, the extracted pattern (. . . 5.1..) is included into the offsprings

rules are incorporated into the crossover operator of an evolutionary algorithm to reduce the search space for the offsprings (Fig. 32). The obtained results indicate that those learnable evolution models allow to speedup the search and improve the quality of solutions (Michalski 2000).

EDA (Estimation of Distribution Algorithms can also be considered as LTH hybrids using estimated probability distributions to generate new solutions. Similarly, *cultural algorithms* use high quality individuals to develop beliefs constraining the way in which individuals are transformed by genetic operators (Reynolds et al. 2005). In cultural algorithms, beliefs are formed based on each entity's individual experiences. The reasoning behind this is that cultural evolution allows populations to learn and adapt at a rate faster than pure biological evolution. Importantly, the learning which takes place individually by each entity is passed on the remainder of the group, allowing learning to take place at a much faster rate.

Civilized genetic algorithms constitute another LTH hybrid approach integrating concepts from machine learning (Sebag et al. 1997). They differ from Darwinian evolution as they keep information of the population in order to avoid doing the same errors. The knowledge is dynamically updated during the successive generations. They have been applied to binary encodings in which a preprocessing step using a genetic algorithm is carried out to obtain a diversified population.

Parameter setting: A dynamic setting of the parameters of a P-metaheuristic can be carried out by a data mining task. Any parameter of a P-metaheuristic, such as

the mutation and crossover probabilities in evolutionary algorithms, the pheromone update in ant colonies, and the velocity update in particle swarm optimization, can be dynamically modified during the search. Indeed, knowledge extracted during the search may serve to dynamically change at run time the values of those parameters. For instance, the initialization of the mutation rate may be adjusted adaptively by computing the progress of the last applications of the mutation operator (Tuson and Ross 1998; Hong et al. 2000). Hence, it becomes possible to determine the probabilities of application of a given operator in an adaptive manner where the more efficient an operator, the likelier its application. Another approach could be to analyze in details the new individuals generated by operators (in terms of quality and diversity) using clustering algorithms. This would give valuable information that can help to set the new application probabilities.

Optimization model: Many optimization problems such as engineering design problems are concerned by expensive objective functions. In this hybrid scheme, supervised classification algorithms can be used to approximate the objective function during the search. The number of solutions to evaluate according to the real objective function can also be reduced. In this case, already evaluated solutions will represent the predefined classes. A non evaluated solution is classified, using for example the k-nearest neighbor classification algorithm. The objective function of a given solution is then approximated using the evaluated solution of the associated class.

This process may also be carried out by clustering algorithms using *fitness imitation*. A clustering algorithm is applied on a population of solutions to be evaluated. Each cluster will have a representative solution. Only the solution that represents the cluster is evaluated (Rasheed et al. 2002; Kim and Cho 2001; Jin and Sendhoff 2004). Then, the objective function of other solutions of the cluster is estimated with respect to its associated representative[11] (Fig. 33). Different clustering techniques may be used such as K-means and fuzzy C-means.

5.2.3 High-level relay hybrid (HRH)

In this HRH hybrid approach, a priori knowledge is first extracted from the target optimization problem. Then, this knowledge is used into the metaheuristic for a more efficient search. The previously acquired knowledge may be obtained from previous experimentations, an expert, landscape analysis, etc. Many schemes may be introduced into this traditional hybrid class.

Search components: For instance, data mining algorithms may be applied for the initialization of solutions. Instead of generating the initial solutions randomly, problem knowledge can be used to generate solutions which integrate "good" patterns.

Example 17 (Any-time learning algorithm in dynamic optimization) A genetic algorithm has been initialized with a case-based reasoning in a tracker/target simulation with a periodically changing environment (Ramsey and Grefenstette 1993). Case-based initialization (learning agent) allows the system to automatically bias the search

[11] This scheme is called fitness imitation or fitness inheritance.

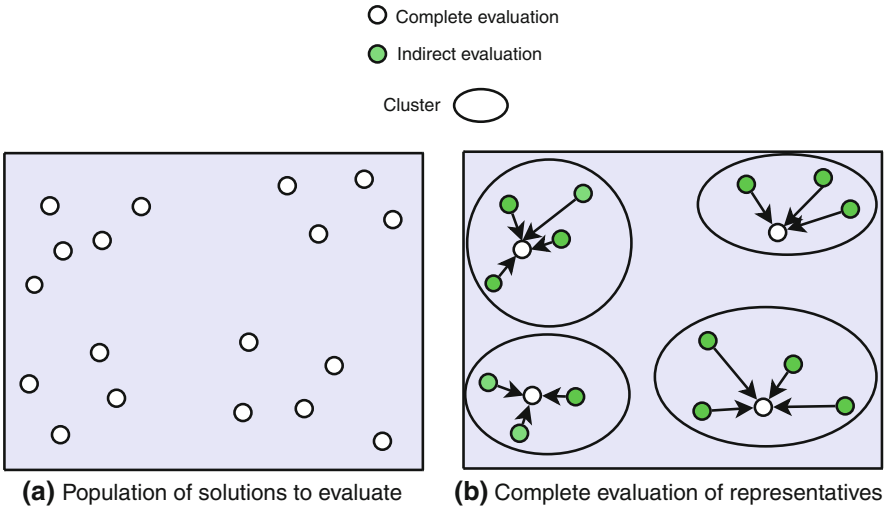
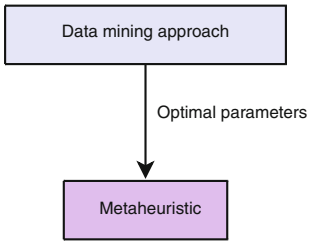


Fig. 33 Evaluating a solution by using the representative of its cluster (fitness imitation)

Fig. 34 Setting the parameters of a metaheuristic using a data mining approach



of the GA toward relevant areas of the search space in a changing environment (dynamic optimization problem). This scheme may be seen as a general approach to continuous learning in a changing environment. The learning agent continuously tests search strategies using different initial solutions. This process allows the update of the knowledge base on the basis of the obtained results. This knowledge base generated by a simulation model will be used by any search agent.

Parameter setting: The same hybrid scheme may be used within the initialization of the parameters of any metaheuristic. A difficult part in designing metaheuristics deals with the setting of their parameters. Indeed, many parameters compose metaheuristics such as the probability of application of a given operator, the tabu list, the size of the population or the number of iterations. An empirical approach consists in both running several times the metaheuristic with different parameters values and trying to select the best values. If the number of trials or the number of parameters is important, determining the best set of parameters may require some statistical analysis. This may be seen as a data mining help (Fig. 34).

Optimization model: Data mining techniques can also be used in decomposing the optimization problem handled by a metaheuristic. For instance, in optimization

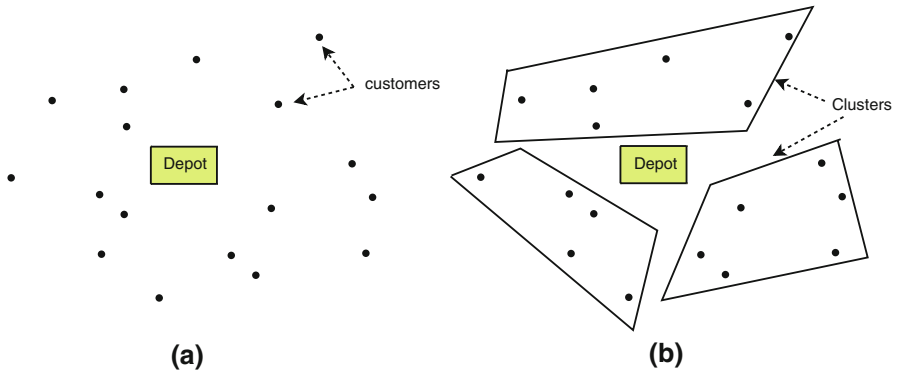


Fig. 35 Decomposing an optimization problem using clustering algorithms. **a** Instance of the VRP problem. **b** Clustering the customers and then applying a TSP metaheuristic to the subproblems

problems dealing with Euclidean distances, such as vehicle routing and the P-median optimization problems, clustering algorithms may be used to decompose the input space into subspaces. Metaheuristics are then used to solve those subproblems associated to the subspaces. Finally, a global solution is built using partial final solutions.

Example 18 (Clustering routing problems) Some efficient techniques in solving routing problems (e.g. TSP, VRP) decompose the operational space into subspaces using clustering algorithms such as the K-means or the EM (Expectation Maximization) algorithm (Fig. 35). Indeed, a metaheuristic is then used to solve the different subproblems. This approach is interesting for very large problems instances.

A popular off-line hybrid scheme for expensive objective function consists in approximating the objective function of the problem. Indeed, in many complex real-life applications, the objective function is very expensive to compute. The main objective of this hybrid approach is to improve the efficiency of the search. The approximation can be used either for expensive objective functions or multi-modal functions. A comprehensive survey on objective function approximations may be found in Jin (2005). Data mining approaches are used to build approximate models of the objective function. In this context, previously evaluated solutions are learned by a data mining algorithm to approximate the objective function of other individuals (Fig. 36). Many learning algorithms may be used such as neural networks (e.g. multi-layer perceptrons, radial-basis-function networks). The main issue here is to obtain a "good" approximation in terms of maximizing the quality and minimizing the computing time. Many questions arise in the design of this hybrid scheme such as: which proportion of the visited solutions are evaluated using the approximation, and at what time or in which component of the search algorithm the approximation is used.

5.2.4 High-level teamwork hybrid (HTH)

A HTH approach is a hybrid scheme in which a dynamically acquired knowledge is extracted in parallel during the search in cooperation with a metaheuristic (Fig. 37).

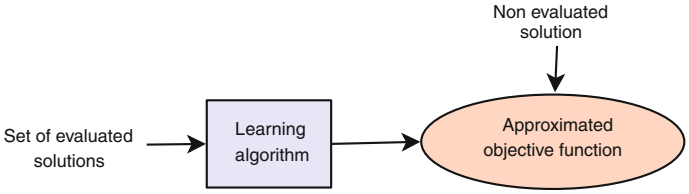


Fig. 36 Data mining approach to approximate the objective function

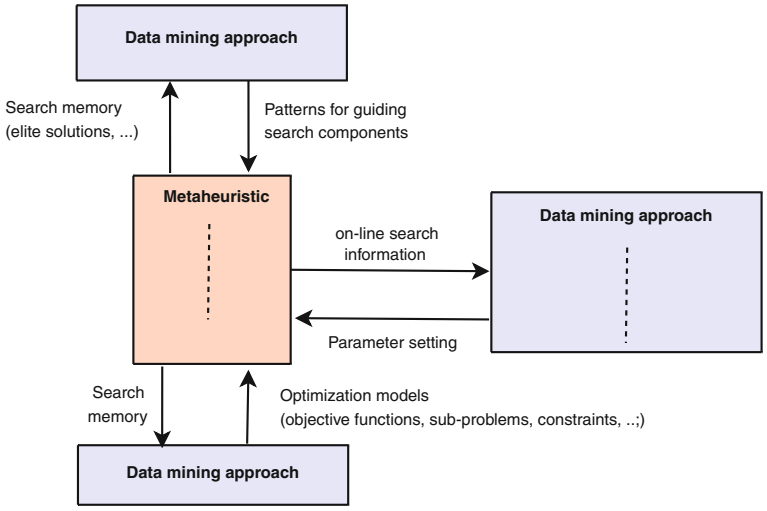


Fig. 37 On-line knowledge extraction and its use by a metaheuristic

Any on-line learning algorithms can be used to extract knowledge from informations provided by metaheuristics such as elite solutions, diversified set of good solutions, frequency memory, recency memory, etc. From this input, the data mining agent extracts useful information to be used by metaheuristics to improve the search. Any statistical indicator for landscape analysis of a problem may also be used.

Example 19 (Data mining in population management of a P-metaheuristic) Using the same scheme of cooperation, data mining approaches can be used to manage the population of a P-metaheuristic. Managing a population deals with the intensification and the diversification tasks of a metaheuristic. Diversification may be carried out by injecting new individuals into the population during the search. In order to lead the search to promising search spaces it could be also interesting to regularly introduce individuals that are built based on information of the past encountered high-quality solutions.

Such an approach has been proposed in the CIGAR (Case Injected Genetic AlgoRithm) algorithm (Louis 2003). The aim of CIGAR is to periodically provide to the genetic algorithm solutions that suit to similar instances/problems. Hence, a classification task is carried out to find similar instances in a case base. CIGAR has been successfully applied to several problems such as the job-shop scheduling and circuit

modeling. For instance, a combination of a GA with the A-priori algorithm has been used to discover interesting subroutines for the oil collecting vehicle routing problem (Dalboni et al. 2003). The obtained sub-routes are inserted into the new individuals of the population. Another illustrative example is the combination of the GRASP heuristic with A-priori like algorithms to extract promising patterns from elite solutions (Ribeiro et al. 2006).

6 Conclusions and perspectives

The efficient solution of complex problems generally involves ideas from different paradigms: metaheuristics, mathematical programming, constraint programming, machine learning, graph theory, parallel and distributed computing, and so on. Pure metaheuristics are not generally well suited to search in high-dimensional and complex landscapes. Hybrid metaheuristics represent actually the most efficient algorithms for many classical and real-life difficult problems. This is shown by the huge number of efficient hybrid metaheuristics proposed to solve a large variety of problems.

Nowadays, combining metaheuristics has become a common strategy to solve optimization problems. Hybrid algorithms will constitute competitive candidates for solving difficult optimization problems in the future years. As we have developed a unified view of metaheuristics which is based on their key search components, one can say that designing a metaheuristic can be reduced to select the most suited search components and combining them. This design approach is naturally a hybrid one, and it is not under the control of a single paradigm of metaheuristics.[12]

A unified taxonomy, based on a hierarchical (low level versus high level, relay versus teamwork) and flat classification (homogeneous/heterogeneous, global/partial, general/specialist), has been developed to describe in terms of design and implementation the different hybridization schemes of metaheuristics with:

- **Metaheuristics:** Combining P-metaheuristics with S-metaheuristics has provided very powerful search algorithms. Pure P-metaheuristics such as evolutionary algorithms, scatter search, and ant colonies are generally not well suited to fine-tuned search in highly combinatorial spaces. P-metaheuristic are more efficient in terms of diversification (i.e. exploration) in the search space. Hence, they need to be combined with more intensification-based (i.e. exploitation-based) search algorithms which are generally based on S-metaheuristics (e.g. local search, tabu search).
- **Mathematical programming:** In the last decade, there have been important advances in designing efficient exact methods in the operations research community (e.g. integer programming). There are many opportunities to design hybrid approaches combining metaheuristics and exact methods. Indeed, the two approaches have complementary advantages and disadvantages (e.g. efficiency and effectiveness).
- **Constraint programming:** Over the last years, interest on combining metaheuristics and constraint programming has risen considerably. The availability of

[12] Using this design approach, it is worthwhile to speak about hybrid metaheuristics as any metaheuristic will be a hybrid one!.

high-level modeling languages and software solvers for constraint programming will lead to more hybrid approaches which capture the most desirable features of each paradigm.

– **Data mining:** Nowadays, using metaheuristics to solve data mining and machine learning problems becomes common. But the challenge is the incorporation of machine learning and data mining techniques into metaheuristics. The major interest in using machine learning and data mining techniques is to extract useful knowledge from the history of the search in order to improve the efficiency and the effectiveness of metaheuristics. Both positive and negative knowledge must be extracted. In fact, most of the actual works focus only on positive knowledge (Minsky 1994).

The main drawback of hybridization is the introduction of new parameters which define the hybrid scheme. The setting of those parameters is non trivial. A crucial question that has to be addressed in the future is an aid for the efficient design of hybrid metaheuristics, in which the automatic setting of parameters must be investigated (Fonlupt et al. 1999; Boese et al. 1994). Indeed, it will be interesting to guide the user to define the suitable hybrid scheme to solve a given problem. It will be also interesting to define "adaptive" cooperation mechanisms which allows to select dynamically the optimization methods according to convergence or other criteria such as diversity. Some approaches such as the COSEARCH (Talbi and Bachelet 2006) or "hyper-heuristics" (Burke et al. 2003) have been proposed to deal with this problem. Those approaches are dedicated to choose the right heuristic for the right operation at the right time during the search. It must be noted that those hybrid approaches operate in the heuristic space, as opposed to most implementations of meta-heuristics, which operate in the solution space. This principle is relatively new, although the concept of "optimizing heuristics" is not a recent one.

Using the software framework ParadisEO (Talbi 2009), it is natural to combine metaheuristics which have been developed under the framework to design S-metaheuristics (under ParadisEO-MO), P-metaheuristics (under ParadisEO-EO), and multi-objective metaheuristics (under ParadisEO-MOEO). Some work still needs to be done in order to combine metaheuristics with exact optimization and machine learning algorithms. The coupling of software frameworks dealing with the three classes of algorithms (i.e. metaheuristics, exact and machine learning algorithms) is an important issue for the future. This enables to reduce the complexity of designing and implementing hybrid approaches and makes them more and more popular.

It will be also interesting to explore parallel models for hybrid methods. Parallel schemes ideally provide novel ways to design and implement hybrid algorithms by providing parallel models of the algorithms. Hence, instead of merely parallelizing and finely tuning a sequential hybrid algorithm which has limited capabilities to be parallelized, teamwork hybrid schemes are inherently suited to parallel environments.

References

Abbattista F, Abbattista N, Caponetti L (1995) An evolutionary and cooperative agent model for optimization. In: IEEE international conference on evolutionary computation ICEC'95, Perth, Australia, pp 668–671

Abramson D, Logothetis P, Postula A, Randall M (1997) Application specific computers for combinatorial optimisation. In: Australien computer architecture workshop, Sydney, Australia

Abramson DA (1992) A very high speed architecture to support simulated annealing. IEEE Comput 25:27–34

Aggarwal CC, Orlin JB, Tai RP (1997) An optimized crossover for the maximum independent set. Oper Res 45:226–234

Aiex RM, Binato S, Ramakrishna RS (2003) Parallel GRASP with path relinking for job shop scheduling. Parallel Comput 29:393–430

Applegate D, Cook W (1991) A computational study of the job-shop scheduling problem. ORSA J Comput 3:149–156

Apt K (2003) Principles of constraint programming. Cambridge University Press, Cambridge

Augerat P, Belenguer JM, Benavent E, Corberan A, Naddef D (1998) Separating capacity constraints in the CVRP using tabu search. Eur J Oper Res 106(2):546–557

Balas E, Niehaus W (1998) Optimized crossover-based genetic algorithms for the maximum cardinality and maximum weight clique problems. J Heuristics 4(2):107–122

Barnhart C, Johnson EL, Nemhauser GL, Savelsbergh MWP, Vance PH (1998) Branch-and-price: column generation for huge integer programs. Oper Res 46:316–329

Beasley JE (1990) OR-library: distributing test problems by electronic mail. J Oper Res Soc 41(11):1069–1072

Belding T (1995) The distributed genetic algorithm revisted. In: Eshelmann D (ed) Sixth international conference on genetic algorithms. Morgan Kaufmann, San Mateo, CA

Belew RK, McInerny J, Schraudolph NN (1991) Evolving networks: using genetic algorithms with connectionist learning. In: Langton CG, Taylor C, Doyne Farmer JD, Rasmussen S (eds) Second confernce on artificial life. Addison-Wesley, USA, pp 511–548

Bellman R (1957) Dynamic programming. Princeton University Press, Princeton, NJ

Benders JF (1962) Partitioning procedures for solving mixed-variables programming problems. Numer Math 4:238–252

Bertsekas DP (1998) Network optimization: continuous and discrete models. Athena Scientific, Belmont, MA

Boese KD (1996) Models for iterative global optimization. PhD thesis, University of California, Los Angeles

Boese KD, Kahng AB, Muddu S (1994) New adaptive multi-start techniques for combinatorial global optimizations. Oper Res Lett 16(2):101–113

Braun H (1990) On solving traveling salesman problems by genetic algorithms. In: Schwefel H-P, Manner R (eds) Parallel problem solving from nature, vol 496 of LNCS. Springer, Dortmund, pp 129–133

Burke EK, Cowling PI, Keuthen R (2001) Effective local and guided variable neighborhood search methods for the asymmetric traveling salesman problem. In: EvoWorkshop. LNCS 2037, Springer, Berlin, pp 203–312

Burke EK, Kendall G, Newall J, Hart E, Ross P, Schulemburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer Academic Publishers, Dordrecht

Caseau Y, Laburthe F (1995) Disjunctive scheduling with task intervals. Technical report LIENS-95-25, Ecole Normale Supérieure de Paris, France

Caseau Y, Laburthe F (1999) Heuristics for large constrained routing problems. J Heuristics 5:281–303

Cesta A, Cortellessa G, Oddi A, Policella N, Susi A (2001) A constraint-based architecture for flexible support to activity scheduling. Lect Notes Comput Sci 2175:369–390

Chabrier A, Danna E, Le C (2002) Pape. Coopération entre génération de colonnes sans cycle et recherche locale appliquée au routage de véhicules. In: Huitièmes Journées Nationales sur la résolution de Problèmes NP-Complets JNPC'2002, Nice, France

Chelouah R, Siarry P (2004) A hybrid method combining continuous tabu search and Nelder–Mead simplex algorithms for the global optimization of multiminima functions. Eur J Oper Res 161(3):636–654

Chen H, Flann NS (1994) Parallel simulated annealing and genetic algorithms: a space of hybrid methods. In: Davidor Y, Schwefel H-P, Manner R (eds) Third conference, on parallel problem solving from nature. Springer, Jerusalem, pp 428–436

Chu PC (1997) A genetic algorithm approach for combinatorial optimization problems. PhD thesis, University of London, London, UK

Chvatal V (1979) A greedy heuristic for the set covering problem. Math Oper Res 4(3):233–235

Clearwater SH, Hogg T, Huberman BA (1992) Cooperative problem solving. In: Huberman BA (ed) Computation: the micro and the macro view. World Scientific, Singapore, pp 33–70
Clearwater SH, Huberman BA, Hogg T (1991) Cooperative solution of constraint satisfaction problems. Science 254:1181–1183
Cohoon J, Hedge S, Martin W, Richards D (1987) Punctuated equilibria: a parallel genetic algorithm. In: Grefenstette JJ (ed) Second international conference on genetic algorithms. MIT, Cambridge, MA, pp 148–154
Cohoon JP, Martin WN, Richards DS (1990) Genetic algorithms and punctuated equilibria. In: Schwefel H-P, Manner R (eds) Parallel problem solving from nature, vol 496. Springer, Dortmund, pp 134–141
Cohoon JP, Martin WN, Richards DS (1991) A multi-population genetic algorithm for solving the k-partition problem on hypercubes. In: Belew RK, Booker LB (eds) Fourth international conference on genetic algorithms. Morgan Kaufmann, San Mateo, CA, pp 244–248
Cook W, Seymour P (2003) Tour merging via branch-decomposition. INFORMS J Comput 15(3):233–248
Crainic TG, Nguyen AT, Gendreau M (1997) Cooperative multi-thread parallel tabu search with evolutionary adaptive memory. In: 2nd international conference on metaheuristics, Sophia Antipolis, France
Crainic TG, Toulouse M, Gendreau M (1995) Synchronous tabu search parallelization strategies for multicommodity location-allocation with balancing requirements. OR Spektrum 17:113–123
Crainic TG, Toulouse M (2003) Parallel strategies for metaheuristics. In: Glover FW, Kochenberger GA (eds) Handbook of metaheuristics. Springer, Berlin, pp 475–513
Cung V-D, Mautor T, Michelon P, Tavares A (1997) A scatter search based approach for the quadratic assignment problem. In: IEEE international conference on evolutionary computation ICEC'97. Indianapolis, USA
Dalboni FL, Ochi LS, Drummond LMD (2003) On improving evolutionary algorithms by using data mining for the oil collector vehicle routing problem. In: International network optimization conference INOC'2003, Paris, France
Davis L (1985) Job-shop scheduling with genetic algorithms. In: Grefenstette JJ (ed) International conference on genetic algorithms and their applications, Pittsburgh, pp 136–140
Dimitrescu I, Stutzle T (2003) Combinations of local search and exact algorithms. In: Evo workshops, pp 211–223
Dowsland KA (1998) Nurse scheduling with tabu search and strategic oscillation. Eur J Oper Res 106:393–407
Dowsland KA, Herbert EA, Kendall G (2006) Using tree search bounds to enhance a genetic algorithm approach to two rectangle packing problems. Eur J Oper Res 168(2):390–402
Dowsland KA, Thomson JM (2000) Solving a nurse scheduling problem with knapsacks, networks and tabu search. J Oper Res Soc 51:825–833
Eby D, Averill R, Punch W, Goodman E (1998) Evaluation of injection island model GA performance on flywheel design optimization. In: International conference on adaptive computing in design and manufacturing. Springer, Devon, pp 121–136
Engelmore RS, Morgan A (1988) Blackboard systems. Addison-Wesley, Reading, MA
De Falco I, Del Balio R, Tarantino E (1997) An analysis of parallel heuristics for task allocation in multicomputers. Computing 59(3):259–275
De Falco I, Del Balio R, Tarantino E, Vaccaro R (1994) Improving search by incorporating evolution principles in parallel tabu search. In: IEEE conference on evolutionary computation, pp 823–828
Federgruen A, Tzur M (1999) Time-partitioning heuristics: application to one warehouse, multi-item, multiretailer lot-sizing problems. Nav Res Logist 46:463–486
Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
Feo TA, Resende MGC, Smith SH (1994) A greedy randomized adaptive search procedure for maximum independent set. Oper Res 42:860–878
Feo TA, Venkatraman K, Bard JF (1991) A GRASP for a difficult single machine scheduling problem. Comput Oper Res 18:635–643
Filho, GR, Lorena LAN (2000) Constructive genetic algorithm and column generation: an application to graph coloring. In: APORS'2000 conference of the Association of the Asian-Pacific Operations Research Societies within IFORS
Fischetti M, Lodi A (2003) Local branching. Math Program 98(1–3):23–47
Fisher ML (1985) An application oriented guide to lagrangian relaxation. Interfaces 15:399–404
Fleurent C, Ferland JA (1994) Genetic hybrids for the quadratic assignment problem. DIMACS Ser Discret Math Theor Comput Sci 16:173–188

Fleurent C, Ferland JA (1996) Genetic and hybrid algorithms for graph coloring. Ann Oper Res 63(3):437–461

Focacci F, Laburthe F, Lodi A (2002) Local search and constraint programming. International Series in Operations Research and Management Science. In: Ribeiro CC, Hansen P (eds) Handbook of metaheuristics. Kluwer Academic Publishers, Norwell, MA

Fonlupt C, Robillard D, Preux P, Talbi E-G (1999) Fitness landscape and performance of metaheuristics. In: Meta-heuristics—advances and trends in local search paradigms for optimization. Kluwer Academic Press, Dordrecht, pp 255–266

Gilmore PC, Gomory RE (1961) A linear programming approach to the cutting stock problem. Oper Res 9:849–859

Ginsberg ML (1993) Dynamic backtracking. J Artif Intell Res 1:25–46

Golden B, Pepper J, Vossen T (1998) Using genetic algorithms for setting parameter values in heuristic search. Intell Eng Syst Through Artif Neural Netw 1:9–32

Gomory RE (1958) Outline of an algorithm for integer solutions to linear programs. Bull AMS 64: 275–278

Grefenstette JJ (1987) Incorporating problem specific knowledge into genetic algorithms. In: Davis L (ed) Genetic algorithms and simulated annealing, Research Notes in Artificial Intelligence. Morgan Kaufmann, San Mateo, CA, pp 42–60

Gutin GM (1999) Exponential neighborhood local search for the traveling salesman problem. Comput Oper Res 26(4):313–320

Habet D, Li CM, Devendeville L, Vasquez M (2002) A hybrid approach for SAT. In: CP'2003 principles and practice of constraint programming. LNCS No. 2470, Springer, Ithaca, USA, pp 172–184

Hansen P, Mladenovic M, Perez-Britos D (2001) Variable neighborhood decomposition search. J Heuristics 7(4):330–350

Hart WE (1994) Adaptive global optimization with local search. PhD thesis, University of California, San Diego

Harvey WD, Ginsberg ML (1997) Limited discrepancy search. In: IJCAI international joint conference on artificial intelligence. Morgan Kaufmann, Burlington, MA, pp 607–613

Hindi KS, Fleszar K, Charalambous C (2003) An effective heuristic for the CLSP with setup times. J Oper Res Soc 54:490–498

Hogg T, Williams C (1993) Solving the really hard problems with cooperative search. In: 11th conference on artificial intelligemce AAAI'93. AAAI Press, Menlo Park, CA, pp 231–236

Hong T-P, Wang H-S, Chen W-C (2000) Simultaneous applying multiple mutation operators in genetic algorithm. J Heuristics 6(4):439–455

Huberman BA (1990) The performance of cooperative processes. Physica D 42:38–47

Husbands P, Mill F, Warrington S (1990) Genetic algorithms, production plan optimisation and scheduling. In: Schewefel H-P, Manner R (eds) Parallel problem solving from nature, vol 496 of LNCS. Springer, Dortmund, pp 80–84

Jahuira CAR, Cuadros-Vargas E (2003) Solving the TSP by mixing GAs with minimal spanning trees. In: First international conference of the Peruvian Computer Society, Lima, Peru, pp 123–132

Jin Y (2005) A comprehensive survey of fitness approximation in evolutionary computation. Soft Comput 9(1):3–12

Jin Y, Sendhoff B (2004) Reducing fitness evaluations using clustering techniques and neural network ensembles. In: Genetic and evolutionary computation GECCO'2004, LNCS No. 3102. Springer, Berlin, pp 688–699

Jog P, Suh JY, Van Gucht D (1989) The effects of population size, heuristic crossover and local improvement on a genetic algorithm for the traveling salesman problem. In: 3rd international conference on genetic algorithms. Morgan Kaufmann, Burlington, MA

Jourdan L, Basseur M, Talbi E-G (2009) Hybridizing exact methods and metaheuristics: a taxonomy. Eur J Oper Res 199(3):620–629

Jourdan L, Dhaenens C, Talbi E-G (2006) Using data mining techniques to help metaheuristics: a short survey. In: Hybrid metaheuristics (HM'2006), vol 4030 of LNCS, Gran Canaria, Spain, pp 57–69

Juenger M, Reinelt G, Thienel S (1995) Practical problem solving with cutting plane algorithms in combinatorial optimization. DIMACS Ser Discret Math Theor Comput Sci 20:111–152

Kamarainen O, Sakkout HE (2002) Local probing applied to scheduling. In: CP'2002 international conference on principles and practice of constraint programming, pp 155–171

Karp RM (1977) Probabilistic analysis of partitioning algorithms for the traveling salesman problem in the plane. Math Oper Res 2:209–224
Kim H, Hayashi Y, Nara K (1995) The performance of hybridized algorithm of genetic algorithm simulated annealing and tabu search for thermal unit maintenance scheduling. In: 2nd IEEE conference on evolutionary computation ICEC'95, Perth, Australia, pp 114–119
Kim H-S, Cho S-B (2001) An efficient genetic algorithm with less fitness evaluation by clustering. In: Congress on evolutionary computation CEC'01. IEEE Press, New York, pp 887–894
Kostikas K, Fragakis C (2004) Genetic programming applied to mixed integer programming. In: Keijzer M et al. (eds) EuroGP conference on genetic programming. LNCS vol 3003, Springer, Berlin, pp 113–124
Koza J, Andre D (1995) Parallel genetic programming on a network of transputers. Technical report CS-TR-95-1542, Stanford University
Krueger M (1993) Méthodes d'analyse d'algorithmes d'optimisation stochastiques à l'aide d'algorithmes génétiques. PhD thesis, Ecole Nationale Supèrieure des Télécommunications, Paris, France
Levine D (1994) A parallel genetic algorithm for the set partitioning problem. PhD thesis, Argonne National Laboratory, Illinois Institute of Technology, Argonne, USA
Lin FT, Kao CY, Hsu CC (1991) Incorporating genetic algorithms into simulated annealing. In: Proceedings of the fourth international symposium on AI, pp 290–297
Louis SJ (2003) Genetic learning from experiences. In: Congress on evolutionary computations CEC'2003, Australia, pp 2118–2125
Lourenco HR (1995) Job-shop scheduling: computational study of local search and large-step optimization methods. Eur J Oper Res 83:347–367
Mahfoud SW, Goldberg DE (1995) Parallel recombinative simulated annealing: a genetic algorithm. Parallel Comput 21:1–28
Maniezzo V (1999) Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. INFORMS J Comput 11(4):358–369
Mariano CE, Morales E (1998) A multiple objective ant-q algorithm for the design of water distribution irrigation networks. In: First international workshop on ant colony optimization ANTS'98, Brussels, Belgium
Martin OC, Otto SW, Felten EW (1992) Large-step markov chains for the TSP: incorporating local search heuristics. Oper Res Lett 11:219–224
Mautor T, Michelon P (1997) Mimausa: a new hybrid method combining exact solution and local search. In: Second international conference on metaheuristics, Sophia-Antipolis, France
Michalski RS (2000) Learnable evolution model: evolutionary processes guided by machine learning. Mach Learn 38(1):9–40
Minsky M (1994) Negative expertise. Int J Expert Syst 7(1):13–19
Nagar A, Heragu SS, Haddock J (1995) A metaheuristic algorithm for a bi-criteria scheduling problem. Ann Oper Res 63:397–414
Narayek A, Smith S, Ohler C (2003) Integrating local search advice into a refinment search solver (or not). In: CP'03 Workshop on cooperative constraint problem solvers, pp 29–43
Nemhauser G, Wolsey L (1999) Integer and combinatorial optimization. Wiley, New York
Nissen V (Jan 1994) Solving the quadratic assignment problem with clues from nature. IEEE Trans Neural Netw 5(1):66–72
Nuijten W, Le Pape C (1998) Constraint based job scheduling with ILOG scheduler. J Heuristics 3:271–286
Nwana V, Darby-Dowman K, Mitra G (2005) A cooperative parallel heuristic for mixed zero-one linear programming. Eur J Oper Res 164:12–23
O'Reilly U-M, Oppacher F (1995) Hybridized crossover-based techniques for program discovery. In: IEEE international conference on evolutionary computation ICEC'95, Perth, Australia, pp 573–578
Patterson R, Rolland E, Pirkul H (1999) A memory adaptive reasoning technique for solving the capacitated minimum spanning tree problem. J Heuristics 5:159–180
Pesant G, Gendreau M (1999) A view of local search in constraint programming. J Heuristics 5:255–279
Potts CN, Velde SL (1995) Dynasearch—iterative local improvement by dynamic programming. Technical Report TR, University of Twente, Netherlands
Prestwich S (2002) Combining the scalability of local search with the pruning techniques of systematic search. Ann Oper Res 115:51–72
Puchinger J. Raidl GR (2005) Combining metaheuristics and exact algorithms in combinatorial optimization: a survey and classification. In: Artificial intelligence and knowledge engineering applications: a bioinspired approach. LNCS vol 3562, Springer, Berlin, pp 41–53

Ramsey CL, Grefenstette JJ (1993) Case-based initialization of genetic algorithms. In: Fifth international conference on genetic algorithms, pp 84–91
Rasheed K, Vattam S, Ni X (2002) Comparison of methods for developing dynamic reduced models for design optimization. In: CEC'2002 congress on evolutionary computation, pp 390–395
Reynolds RG, Michalewicz Z, Peng B (2005) Cultural algorithms: computational modeling of how cultures learn to solve problems—an engineering example. Cybern Syst 36(8):753–771
Ribeiro M, Plastino A, Martins S (2006) Hybridization of GRASP metaheuristic with data mining techniques. J Math Modell Algorithms 5(1):23–41
Rosing KE, ReVelle CS (1997) Heuristic concentration: two stage solution construction. Eur J Oper Res 97(1):75–86
Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by backpropagating errors. Nature 323:533–536
Salami M, Cain G (1996) Genetic algorithm processor on reprogrammable architectures. In: Fifth annual conference on evolutionary programming EP'96. MIT Press, San Diego, CA
Sebag M, Schoenauer M, Ravise C (1997) Toward civilized evolution: developing inhibitions. In: Bäck T (eds) Seventh international conference on genetic algorithms, pp 291–298
Sefraoui M, Periaux J (2000) A hierarchical genetic algorithm using multiple models for optimization. In: Parallel problem solving from nature PPSN'2000, LNCS No.1917. Springer, Berlin, pp 879–888
Sellmann M, Ansótegui C (2006) Disco-novo-gogo: integrating local search and complete search with restarts. In: The twenty-first national conference on artificial intelligence and the eighteenth innovative applications of artificial intelligence conference, Boston, USA
Shahookar K, Mazumder P (1990) A genetic approach to standard cell placement using meta-genetic parameter optimization. IEEE Trans Comput Aided Design 9(5):500–511
Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M, Puget J-F (eds) CP'98 principle and practice of constraint programming, LNCS No. 1520, pp 417–431
Sprave J (1999) A unified model of non-panmictic population structures in evolutionary algorithms. In: Proceedings of the 1999 congress on evolutionary computation, vol 2. IEEE Press, Piscataway, NJ, pp 1384–1391
Stutzle T, Hoos HH (1997) The MAX-MIN ant system and local search for combinatorial optimization problems: towards adaptive tools for global optimization. In: 2nd international conference on metaheuristics. INRIA, Sophia Antipolis, France, pp 191–193
Suh JY, Van Gucht D (1987) Incorporating heuristic information into genetic search. In: 2nd international conference on genetic algorithms. Lawrence Erlbaum Associates, USA, pp 100–107
Taillard E (1993) Parallel iterative search methods for vehicle routing problem. Networks 23:661–673
Taillard E (2003) Heuristic methods for large centroid clustering problems. J Heuristics 9(1):51–74
Taillard E, Voss S (2002) POPMUSIC: partial optimization metaheuristic under special intensification conditions. In: Essays and surveys in metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 613–629
Taillard ED, Gambardella L (1997) Adaptive memories for the quadratic assignment problem. Technical Report 87–97, IDSIA, Lugano, Switzerland
Taillard ED, Gambardella LM, Gendreau M, Potvin J-Y (2001) Adaptive memory programming: a unified view of metaheuristics. Eur J Oper Res 135(1):1–16
Talbi E-G (2002) A taxonomy of hybrid metaheuristics. J Heuristics 8:541–564
Talbi E-G (2009) Metaheuristics: from design to implementation. Wiley, New York
Talbi E-G, Bachelet V (2006) COSEARCH: a parallel cooperative metaheuristic. J Math Modell Algorithms 5(2):5–22
Talbi E-G, Fonlupt C, Preux P, Robillard D (1998) Paysages de problèmes d'optimisation et performances des méta-heuristiques. In: Premier Congrés de la Société Francaise de Recherche Opérationnelle et Aide à la Décision ROAD, Paris, France
Talbi EG, Muntean T, Samarandache I (1994) Hybridation des algorithmes génétiques avec la recherche tabou. In: Evolution Artificielle EA94, Toulouse, France
Talukdar S, Baerentzen L, De Gove A, Souza P (1998) Asynchronous teams: cooperation schemes for autonomous agents. J Heuristics 4(4):295–321
Tamura H, Hirahara A, Hatono I, Umano M (1994) An approximate solution method for combinatorial optimization—hybrid approach of genetic algorithm and lagrangean relaxation method. Trans Soc Instrum Control Eng 130:329–336

Tanese R (1987) Parallel genetic algorithms for a hypercube. In: Proceedings of the second international conference on genetic algorithms. MIT, Cambridge, MA, pp 177–183

Thiel J, Voss S (1994) Some experiences on solving multiconstraint zero-one knapsack problems with genetic algorithms. INFOR 32(4):226–242

Toulouse M, Crainic T, Gendreau M (1996) Communication issues in designing cooperative multi-thread parallel searches. In: Osman IH, Kelly JP (eds) Meta-heuristics: theory and applications. Kluwer Academic Publishers, Dordrecht, pp 501–522

Tuson A, Ross P (1998) Adapting operator settings in genetic algorithms. Evol Comput 6(2):161–184

Ulder NLJ, Aarts EHL, Bandelt H-J, Van Laarhoven PJM, Pesch E (1990) Genetic local search algorithms for the traveling salesman problem. In: Schewefel H-P, Manner R (eds) Parallel problem solving from nature, vol 496 of LNCS. Springer-Verlag, Dortmund, pp 109–116

Vasquez M, Hao J-K (2001) A hybrid approach for the 0–1 multidimensional knapsack problem. In: Proceedings of the international joint conference on artificial intelligence IJCAI, pp 328–333

Verhoeven MGA, Aarts EHL (1995) Parallel local search. J Heuristics 1(1):43–65

Voigt H-M, Born J, Santibanez-Koref I (1990) Modelling and simulation of distributed evolutionary search processes for function optimization. In: Schwefel H-P, Manner R (eds) Parallel problem solving from nature, vol 496 of LNCS. Springer, Dortmund, pp 373–380

Voss S (1993) Tabu search: applications and prospects. In: Du DZ, Pardalos PM (eds) Network optimization problems. World Scientific, USA, pp 333–353

Wang L-H, Kao C-Y, Ouh-young M, Chen W-C (1995) Molecular binding: a case study of the population-based annealing genetic algorithms. In: IEEE international conference on evolutionary computation ICEC'95, Perth, Australia, pp 50–55

Yagiura M, Ibaraki T (1996) Metaheuristics as robust and simple optimization tools. In: IEEE international conference on evolutionary computation, ICEC'96, pp 541–546